

Efficient $(1, s)$ -Nucleus Decomposition on Billion-Scale Graphs

Anonymous Authors
Submission #XXX

Abstract

Extracting cohesive substructures from large graphs is a fundamental task in graph mining, with applications in community detection, dense-subgraph search, and role analysis. The $(1, s)$ -nucleus decomposition is a higher-order generalization of k -core that ranks each vertex by the s -clique witnesses surrounding it (triangles for $s = 3$, 4-cliques for $s = 4$, and so on), giving sharper rankings than the standard k -core but at much higher cost. Existing studies compute $(1, s)$ -nucleus inside a general (r, s) framework that encodes the witnesses with a Clique Path Index but maintains the index as mutable residual state during peeling. However, this mutability causes significant overhead in residual-index rewrites, memory consumption, and per-pop work, limiting scalability to large graphs and high s . We observe that the $(1, s)$ special case has a stronger invariant: vertex peeling preserves every edge among surviving vertices, so every CPI path stays structurally valid throughout the peel. This invariant turns exact decomposition into counter maintenance over a static CPI, which yields SPIN* (a single-pass bucket-queue peel solver), ParaBuild (a parallel CPI constructor, now the dominant phase once peeling is cheap), and BuildHier (a post-peel hierarchy emitter that reuses the preserved index). Experiments on 451 matched successful configurations show that SPIN* achieves a $17.90\times$ geometric-mean peel-time speedup and a $49.48\times$ maximum peel-time speedup over the mutable baseline, while reducing memory usage by a $1.55\times$ geometric-mean ratio; the static-state pipeline also scales to a billion-edge graph for which the mutable baseline completes only at $s=2$.

CCS Concepts

• **Theory of computation** → **Graph algorithms analysis**; • **Information systems** → **Data mining**.

Keywords

nucleus decomposition, clique counting, cohesive subgraphs, hierarchy

ACM Reference Format:

Anonymous Authors. 2027. Efficient $(1, s)$ -Nucleus Decomposition on Billion-Scale Graphs. In *Proceedings of 2027 ACM SIGMOD International Conference on Management of Data (SIGMOD '27)*. ACM, New York, NY, USA, 15 pages. <https://doi.org/XXXXXXX.XXXXXXX>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGMOD '27, June 2027, TBD

© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-XXXX-XXXX-X/27/06...\$15.00
<https://doi.org/XXXXXXX.XXXXXXX>

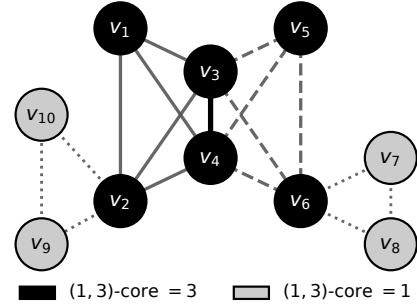


Figure 1: Running example of $(1, 3)$ -nucleus decomposition.

1 Introduction

Core decomposition ranks vertices by the amount of cohesive structure that still surrounds them after weak vertices are removed [5, 41, 48]. It supports a wide range of graph-mining tasks, including community detection [13, 17, 18, 20], dense-subgraph discovery [19], influential-vertex and role analysis [11, 24, 30], network visualization [51], and applications in social, biological, and security networks [3, 33, 34]. The classical k -core uses edges as the witness, but many of these tasks need a stronger witness because a group may pass an edge-degree threshold while containing few triangles, 4-cliques, or larger coordinated patterns [1, 12, 22]. The $(1, s)$ -nucleus decomposition [39, 40] gives this vertex-centric higher-order ranking: for a fixed s , it assigns each vertex the largest k for which the vertex belongs to a maximal region where every vertex participates in at least k internal s -cliques. For $s = 2$ it reduces to the k -core; for larger s it ranks vertices by progressively tighter clique-supported cohesion.

Example 1.1. Figure 1 shows a 10-vertex graph at $s = 3$. The six vertices $v_1, \dots, v_6$ belong to two triangle-rich K_4 blocks; each participates in at least three triangles supported by other vertices of the same value, so all six have $(1, 3)$ -core value 3. The remaining four vertices $v_7, \dots, v_{10}$ each lie in one of two pendant triangles ($\{v_6, v_7, v_8\}$ and $\{v_2, v_9, v_{10}\}$) supporting only one triangle each, so all four have $(1, 3)$ -core value 1. A formal definition is given in Section 2; we revisit this graph as a running example throughout the paper.

Challenges. Computing $(1, s)$ -nucleus decomposition at high s is hard because the number of s -cliques can be enormous; the current state of the art avoids explicit enumeration with the Clique Path Index, or CPI [49]. Using the CPI efficiently for the vertex-centric case $r = 1$ raises four challenges.

Residual state during peeling. The general (r, s) algorithm maintains the CPI as mutable residual state, storing per-path vertex sets, a reverse vertex-to-path index, and a priority queue, and rewriting these structures every time a vertex is popped. At $r=1$, peeling

removes vertices but never deletes edges, so it is not obvious that the same residual maintenance is necessary.

Memory footprint of the residual index. The residual structures dominate memory: per-path vertex hash sets, the reverse vertex-to-path map, and the live Bron-Kerbosch recursion tree together pay constant-factor overhead per vertex-path incidence on top of the clique-encoding cost. A layout whose memory grows only with the unavoidable cost of encoding all s -cliques in the index, and not with hash-table or recursion-tree overhead on top of it, would push the achievable s on a fixed-memory machine to the algorithmic limit.

Construction as the new bottleneck. On dense graphs at high s , CPI construction takes seconds to minutes per (graph, s) configuration and dominates total runtime in any static-index pipeline; parallelising it must avoid the synchronisation cost of mutable path rewrites.

Hierarchy emission from a mutating index. The clique-coherent nucleus hierarchy of Definition 2.3 witnesses s -connectivity through s -cliques contained in each super-level set, so emitting it requires access to the s -cliques themselves. If the CPI is mutated during peeling, those cliques are no longer available afterwards, and the hierarchy needs either a second clique-enumeration pass or per-peel-level state, both of which can dominate total cost.

State-of-the-Art and Limitations. The current state of the art for general (r, s) -nucleus decomposition is CND [49], which maintains the CPI as mutable residual state and inherits four weaknesses when restricted to $r = 1$.

Mutable residual-index maintenance. CND rewrites path records and updates a hash-indexed reverse map at every peel step. These rewrites are a real cost rather than a constant factor: SPIN*, which replaces them with closed-form counter updates over a static index, delivers a 17.90× geometric-mean peel-time speedup over CND on the matched benchmark cells (Section 8).

High memory consumption. The residual structures that CND must keep mutable in the general (r, s) setting (per-path membership queries, reverse vertex-to-path lookup, and the live recursion tree used to rewrite paths) carry roughly an order of magnitude more bytes per encoded vertex-path incidence than a static encoding of the same s -cliques requires (Table 5). The geometric-mean memory ratio CND/SPIN* is 1.55× on the matched cells and 3.92× at maximum; on a billion-scale graph, CND completes only at $s=2$ (using 400 GB of 503 GB) and exceeds the budget for every $s \geq 3$, while SPIN* finishes the full s sweep.

Construction as the residual bottleneck. Even after cheap peeling, CND pays the full construction cost; on web-it-2004 (our highest CPI-build-cost benchmark) at $s=3$, construction takes about 137 seconds and peeling takes about 59 milliseconds, so the static index builder dominates total runtime by more than 2,000× on this configuration. Without a parallel builder, the end-to-end speedup over CND stays at 1.05× on the matched cells even though the peel-time speedup is 17.90×.

Hierarchy emission re-enumerates or replicates. Because CND edits the CPI during peeling, its s -cliques are gone when the hierarchy is requested. The natural workaround, the general (r, s) level-DSU hierarchy specialised to $r=1$, maintains one disjoint-set forest per core level during the peel. It exceeds the available 503 GB on wiki-Talk at $s \geq 9$, where our pipeline finishes (Figure 7). Even where it fits, it

uses up to 43× more memory (gmean 13× over 24 matched cells) than emitting the same hierarchy from a preserved CPI.

Our Contributions. For $(1, s)$ -nucleus decomposition we exploit a static-CPI invariant (Theorem 4.1) to deliver a single pipeline that addresses all four challenges above.

A new $(1, s)$ -nucleus decomposition algorithm. We propose SPIN* (Algorithm 4), a new $(1, s)$ -nucleus decomposition algorithm that computes the core values *without ever modifying the CPI*. A static-CPI invariant (Theorem 4.1) guarantees that every path’s hold and pivot sets stay valid throughout the peel, so SPIN* only maintains a few per-path counters instead of mutating the entire index. SPIN* is on average 17.9× (up to 49×) faster at peeling and noticeably lighter on memory than the prior CND baseline, and it scales to a billion-edge graph that CND cannot complete.

Parallel CPI construction. ParaBuild (Algorithm 5) parallelises CPI construction across seed vertices. This addresses the construction phase that dominates total runtime once peeling is counter-only; empirical scaling reaches 23.4× at $T=24$ threads.

Core-value hierarchy from the static CPI. BuildHier (Algorithm 6) emits the s -clique-connected super-level join tree from the preserved CPI in time near-linear in the static index size, without re-enumerating any s -clique. It uses up to 43× less memory (gmean 13× over 24 matched cells) than the level-DSU baseline.

Experimental evaluation. We compare against CND on ten real-world graphs (451 matched configurations, headline peel speedup 17.90× gmean / 49.48× max, memory-usage ratio 1.55× gmean), report phase behaviour and parallel construction, push the pipeline to a billion-scale graph where CND only completes at $s=2$, and use case studies to evaluate whether the $r=1$ specialization is practically useful.

2 Preliminaries

Graph notation. We work with a simple undirected graph $G = (V, E)$. For a vertex v , $N_G(v) = \{u \in V \mid (u, v) \in E\}$ is its neighbor set and $\deg_G(v) = |N_G(v)|$ is its degree. For $X \subseteq V$, $G[X]$ is the subgraph induced by X . The implementation stores G in CSR form with an offsets array and an edges array.

Definition 2.1 (k -Clique). A k -clique is a vertex set $C \subseteq V$ with $|C| = k$ such that every two distinct vertices in C are adjacent.

Definition 2.2 (s -Clique Support). Given $s \geq 2$, the s -clique support of a vertex v in G is

$$\deg^{(s)}(v) = |\{S \subseteq V \mid v \in S, |S| = s, S \text{ is a clique in } G\}|.$$

$\deg^{(s)}(v)$ is evaluated inside the current induced subgraph when a peel level is being defined; the subscript is dropped when the graph is clear.

Definition 2.3 (k -(1, s)-Nucleus). Let $X \subseteq V$. The induced subgraph $G[X]$ is a k -(1, s)-nucleus if:

- (1) every vertex in X belongs to at least k s -cliques contained in $G[X]$;
- (2) $G[X]$ is s -connected: for any two vertices $u, v \in X$, there is a sequence $u = v_1, \dots, v_t = v$ such that each pair $\{v_i, v_{i+1}\}$ is contained in an s -clique of $G[X]$;
- (3) $G[X]$ is maximal with respect to adding vertices while preserving the first two conditions.

Algorithm 1: BUILD CPI(G, s): standard CPI construction [49].

Input: Undirected graph $G = (V, E)$, clique size s .
Output: Clique Path Index $\mathcal{P}$ of G for clique size s .

```

1  Compute a degeneracy ordering  $(v_1, \dots, v_n)$  of  $G$  and orient every edge
   from lower to higher rank, giving  $N^+(\cdot)$ ;
2   $\mathcal{P} \leftarrow \emptyset$ ;
3  foreach  $i \leftarrow 1$  to  $n$  do
4     $\text{EXPANDPATH}(N^+(v_i), \{v_i\}, \emptyset)$ ; // one root per vertex
5  return  $\mathcal{P}$  Function  $\text{ExpandPath}(P, V_h, V_p)$ :
6    if  $|V_h| > s$  then return;
7    if  $P = \emptyset$  then
8      if  $|V_h| + |V_p| \geq s$  then append the path  $(V_h, V_p)$  to  $\mathcal{P}$ ;
9      return
10    $u_{\text{piv}} \leftarrow \arg \max_{x \in P} |P \cap N^+(x)|$ ; // Tomita pivot
11   foreach  $v \in P \setminus N(u_{\text{piv}})$  do
12      $P \leftarrow P \setminus \{v\}$ ;
13     if  $v = u_{\text{piv}}$  then
14        $\text{ExpandPath}(P \cap N(v), V_h, V_p \cup \{v\})$ ;
15     else
16        $\text{ExpandPath}(P \cap N(v), V_h \cup \{v\}, V_p)$ ;

```

Definition 2.4 (Core Number). Given $s \geq 2$, the $(1, s)$ -nucleus core number of a vertex v , denoted $\kappa_s(v)$, is the largest k for which v belongs to some k -(1, s)-nucleus of G .

The Clique Path Index. The Clique Path Index, or CPI, compactly represents s -cliques without listing them one by one [49].

Definition 2.5 (CPI Path). A CPI *path* is a pair $P = (V_h(P), V_p(P))$ of disjoint vertex sets such that $V_h(P) \cup V_p(P)$ is a clique in G and $|V_h(P)| \leq s \leq |V_h(P)| + |V_p(P)|$. The set $V_h(P)$ is the *hold set*: every encoded s -clique contains all of these vertices. The set $V_p(P)$ is the *pivot set*: an encoded s -clique chooses vertices from this pool. The *pivot requirement* of P is $\eta_P = s - |V_h(P)|$, the number of pivot vertices needed to complete the hold set into an s -clique. The boundary case $\eta_P = 0$ (i.e. $|V_h(P)| = s$) encodes exactly one s -clique, namely the hold set itself; we adopt the convention $\binom{m}{-1} = 0$ so the binomial-difference formulas used later return 0 for any pivot-removal event on such a path. The set of s -cliques encoded by P is

$$C(P) = \{V_h(P) \cup P' \mid P' \subseteq V_p(P), |P'| = \eta_P\}.$$

Its cardinality is $\binom{|V_p(P)|}{\eta_P}$.

The CPI is a set $\mathcal{P}$ of paths such that every s -clique is encoded by exactly one path. We write

$$\Sigma = \sum_{P \in \mathcal{P}} (|V_h(P)| + |V_p(P)|)$$

for the *vertex-path incidence count* of the CPI, i.e. the cost of reading the whole CPI once, in which every runtime and memory bound in this paper is expressed. We also write $\kappa_{\max} = \max_v \deg^{(s)}(v)$. The CPI is built by a degeneracy-ordered Bron-Kerbosch search with Tomita pivoting [9, 15, 44, 49]; we reproduce the construction as Algorithm 1 because the rest of the paper operates over the emitted paths.

Algorithm 1 computes a degeneracy ordering of G and orients edges from lower to higher rank, so each s -clique has a unique smallest-rank owner (Line 1). For every vertex v_i , EXPANDPATH is invoked with hold set $\{v_i\}$ and forward-neighbour candidate set $N^+(v_i)$ (Lines 3-4). A recursive call returns immediately when the hold set already exceeds s (Line 6), and emits a path when the

candidate set is empty and the hold-plus-pivot size meets s (Lines 8-9). Otherwise the Tomita pivot u_{piv} is the vertex of P with the most forward neighbours inside P (Line 10), and every non-neighbour of u_{piv} is recursively expanded: branching on the pivot itself extends the pivot set, branching on any other vertex extends the hold set (Lines 11-16).

Reading clique counts off the CPI. Once the CPI is built, each vertex's exact s -clique support is a sum over the paths through it: every s -clique through v is encoded by exactly one path with v either in the hold or in the pivot set, and the count in each case is a single binomial in the path's pivot size.

Lemma 2.6 (Support Formula [49]). *For every $v \in V$,*

$$\deg^{(s)}(v) = \sum_{P: v \in V_h(P)} \binom{|V_p(P)|}{\eta_P} + \sum_{P: v \in V_p(P)} \binom{|V_p(P)| - 1}{\eta_P - 1}.$$

The first sum is over paths with v in the hold: every choice of η_P pivots from $V_p(P)$ completes $V_h(P)$ into a distinct s -clique through v , giving $\binom{|V_p(P)|}{\eta_P}$ cliques per path. The second sum is over paths with v in the pivot set: v fills one of the η_P pivot slots, and the remaining $\eta_P - 1$ are chosen from the other $|V_p(P)| - 1$ pivots, giving $\binom{|V_p(P)| - 1}{\eta_P - 1}$ cliques per path. Every $\deg^{(s)}(v)$ in the rest of the paper is evaluated through Lemma 2.6, never by listing cliques.

Problem statement. Given G and $s \geq 2$, compute $\kappa_s(v)$ for every $v \in V$; the nucleus hierarchy is constructed afterwards (Section 7).

3 The Mutable-CPI Baseline

CND [49] is the state-of-the-art exact algorithm for general (r, s) -nucleus decomposition. This section walks through its peeling loop, the residual structures it maintains, and the operations this paper specialises away when restricted to $r = 1$.

The peeling loop. CND follows the standard core-decomposition template [5]: it maintains the residual support of every alive r -clique in a priority queue, repeatedly pops the r -clique of minimum residual support, assigns it the current peel level as its core value, and propagates the support loss to the remaining alive r -cliques. At $r=1$ the queue items are vertices, so each iteration pops a vertex v , records $\kappa_s(v)$, and decrements the residual support of every vertex that loses an s -clique because of v 's removal. The non-trivial work happens during this support-loss propagation: the CPI that encodes the s -cliques must be brought into a state consistent with v being gone before the next pop.

Residual state. To keep the support counts consistent across peels, CND maintains the CPI as mutable residual state. For each path P it stores the hold and pivot vertex sets as hash sets so that the test “does v belong to this path?” is constant time. A separate reverse map sends every vertex u to the list of paths that contain it, so when u is popped the algorithm can find the affected paths without scanning all of $\mathcal{P}$. A heap (or bucket queue) keeps every alive vertex ordered by current residual support, with decrease-key operations for the support drops triggered at each peel event. Finally, the Bron-Kerbosch recursion tree that produced the CPI is kept mutable: when a peel event invalidates a path, the algorithm walks into the tree and rewrites or removes the affected nodes.

Per-peel work. A single peel of vertex v costs:

- a reverse-map lookup of v 's path list, then for every such path P a constant-time hash-set query to determine whether v is a hold or a pivot;
- a path rewrite: if v is a hold, the path is deleted from the recursion tree and its reverse-map entries for all other vertices in $V_h(P) \cup V_p(P)$ are erased; if v is a pivot, the path's pivot set shrinks by one and the reverse-map entry for v on this path is erased;
- a support recompute for every path-mate u using the encoded s -clique family that survives the rewrite, followed by a decrease-key on u in the heap.

The dominant components are the path rewrites and the reverse-map erases; together they scale with the number of vertex-path incidences touched by the event, which is what gives CND its observed cost profile.

Why this state is needed in the general case. The mutable design is not over-engineering for the general (r, s) setting. When the peeled object is an edge ($r=2$) or a higher-order clique ($r \geq 3$), the removal can destroy edges among vertices that remain alive, so some of the cliques that a path encodes cease to exist as cliques. In that setting, the stored hold and pivot sets of P no longer define a valid encoded clique family, and the path must be split, shrunk, or deleted by walking back into the recursion tree. The reverse map and the per-path hash sets exist precisely to support that surgery efficiently.

Why this state is wasteful at $r = 1$. The $(1, s)$ case has a stronger invariant: vertex peeling removes vertices but does not delete edges, so a set of vertices that formed a clique when the CPI was built remains a clique throughout the peel (Section 4). A path's encoded clique family therefore stays structurally valid; only the question of whether a particular vertex of $V_h(P) \cup V_p(P)$ is still alive changes. The path rewrites, the reverse-map erases, and the recursion-tree surgery in the per-peel cost above are therefore all unnecessary at $r=1$: they maintain a mutable residual state that is logically equivalent to a static incidence layout plus a single integer counter per path. The next four sections cash in this observation. Section 4 states the static-CPI invariant in counter form; Section 5 derives SPIN* as the peel solver over the resulting static index; Section 6 parallelises construction once peeling becomes counter-only; Section 7 extracts the nucleus hierarchy from the preserved CPI without any clique re-enumeration.

4 Static CPI for $r = 1$

We prove the central invariant of the paper: at $r=1$, vertex peeling never deletes an edge among the surviving vertices, so the stored clique $V_h(P) \cup V_p(P)$ of every path remains structurally valid throughout the peel, and the residual state that CND maintains (Section 3) is unnecessary.

Static structure and dynamic counters. Recall from Section 2 that a path stores the hold set $V_h(P)$, the pivot set $V_p(P)$, and the pivot requirement $\eta_p = s - |V_h(P)|$; none of these three objects is edited during peeling. The dynamic state is only a liveness bit and an integer p_p , the number of pivots in $V_p(P)$ that have not yet been peeled. If a peeled vertex is a hold, every encoded s -clique contains that vertex, and the path contributes nothing after the event; if a peeled vertex is a pivot, the path remains the same symbolic object

and p_p decreases by one. The next theorem records this counter form: a peel event either kills a path contribution or reduces an effective pivot count.

Theorem 4.1 (Vertex Removal on CPI, Counter Form). *Let P be a path of the CPI of G , and let $v \in V$ be removed from G .*

- (1) *If $v \in V_h(P)$, P encodes no surviving s -clique.*
- (2) *If $v \in V_p(P)$, the surviving s -cliques are encoded by the same $V_h(P)$ together with $V_p(P) \setminus \{v\}$. Their count depends only on $|V_p(P)|$. A counter $p_p = |V_p(P)|$ decremented by one is therefore support-equivalent to deleting v from $V_p(P)$. If $p_p - 1 < \eta_p$, no surviving s -clique is encoded.*
- (3) *Otherwise, P is unaffected.*

The dynamic state of P during peeling reduces to an integer counter p_p , the fixed pivot requirement η_p , and a liveness bit. The stored sets $V_h(P)$ and $V_p(P)$ are never modified.

PROOF. P encodes $C(P) = \{V_h(P) \cup P' \mid P' \subseteq V_p(P), |P'| = \eta_p\}$. If $v \in V_h(P)$, every encoded clique contains v and none survives. If $v \in V_p(P)$, the surviving cliques are $\{V_h(P) \cup P' \mid P' \subseteq V_p(P) \setminus \{v\}, |P'| = \eta_p\}$, a family of size $\binom{|V_p(P)|-1}{\eta_p}$ that depends on $V_p(P)$ only through its cardinality; by Lemma 2.6 every support contribution of the path depends only on this cardinality, so the counter update $p_p \leftarrow p_p - 1$ is support-equivalent to physical deletion, and the family is empty iff $p_p - 1 < \eta_p$. If $v \notin V_h(P) \cup V_p(P)$, no encoded clique mentions v and the family is unchanged. $\square$ $\square$

Boundary of the invariant. Theorem 4.1 is specific to $r = 1$. When $r \geq 2$, a peel event removes an edge or a higher-order clique from the residual object. That event can break the clique represented by $V_h(P) \cup V_p(P)$. In that setting, a path may need to be split, shrunk, or rebuilt. The mutable CPI machinery of [49] remains the appropriate general tool, and this paper does not subsume it.

Support loss from one path event. Theorem 4.1 leaves only cardinality changes. The support lost by any other live vertex on the path is therefore a binomial difference.

Lemma 4.2 (Support Loss for a Path Event). *Let P be a live path with pivot count $p = |V_p(P)| \geq \eta$, where $\eta = \eta_p$ is the pivot requirement, and let an event remove $d \geq 1$ live pivots from P . For every other alive vertex u incident to P :*

$$\begin{aligned} \Delta_{\text{hold}}(p, \eta, d) &= \binom{p}{\eta} - \binom{p-d}{\eta} & \text{if } u \in V_h(P), \\ \Delta_{\text{pivot}}(p, \eta, d) &= \binom{p-1}{\eta-1} - \binom{p-d-1}{\eta-1} & \text{if } u \in V_p(P). \end{aligned}$$

A path-killing event, caused by a hold peel or by $p - d < \eta$, is the limiting case in which the entire contribution vanishes: $\binom{p}{\eta}$ for surviving holds and $\binom{p-1}{\eta-1}$ for surviving pivots.

PROOF. By Lemma 2.6, a hold u contributes $\binom{p}{\eta}$ before the event and $\binom{p-d}{\eta}$ after; a pivot u occupies one slot and contributes $\binom{p-1}{\eta-1}$ before and $\binom{p-d-1}{\eta-1}$ after. Subtracting after from before gives the two formulas. $\square$ $\square$

Running example. Figure 1 shows the running example at $s = 3$: two K_4 blocks share edge (v_3, v_4) , and two pendant triangles attach at v_2 and v_6 . The six CPI paths in Table 1 encode the ten triangles through stored V_h, V_p sets and the binomial count $\binom{|V_p|}{s-|V_h|}$. The

Table 1: CPI paths for the running example at $s = 3$.

Path	V_h	V_p	$\eta = s - V_h $	$(V_p \choose \eta)$
P_1	$\{v_7\}$	$\{v_6, v_8\}$	2	1
P_2	$\{v_9\}$	$\{v_2, v_{10}\}$	2	1
P_3	$\{v_1\}$	$\{v_2, v_3, v_4\}$	2	3
P_4	$\{v_2\}$	$\{v_3, v_4\}$	2	1
P_5	$\{v_3\}$	$\{v_4, v_5, v_6\}$	2	3
P_6	$\{v_4\}$	$\{v_5, v_6\}$	2	1
Total				10

(1, 3)-core values fall into two bands: $\kappa_3(v_i) = 3$ for $i \in \{1, \dots, 6\}$ (the two K_4 blocks) and $\kappa_3(v_i) = 1$ for $i \in \{7, \dots, 10\}$ (the pendant triangles); the partition is shown by the node shading in Figure 1. The example exercises both events of Theorem 4.1: a path dies when a hold vertex is removed, and a path's pivot counter shrinks when a pivot vertex is removed.

Example 4.3 (Peel event walk-through). Consider the path $P_3 = (\{v_1\}, \{v_2, v_3, v_4\})$ in Table 1, with $p = 3$ and $\eta = 2$. Peeling v_2 as a pivot triggers Lemma 4.2 with $d = 1$. The surviving hold v_1 loses $\binom{3}{2} - \binom{2}{2} = 2$ from its support. The surviving pivots v_3 and v_4 each lose $\binom{2}{1} - \binom{1}{1} = 1$. The stored set $V_p(P_3)$ is not modified. Only p_{P_3} changes from 3 to 2.

5 Computing Core Values over Static CPI

The static-CPI invariant of Section 4 becomes a concrete algorithm for κ_s through a one-vertex update rule built directly on the encoded s -clique counts (Section 5.1, Algorithm 2). Section 5.3 delivers SPIN* (Algorithm 4), a single-pass bucket-queue solver that assigns core values in nondecreasing peel order through local path-counter updates. Section 5.2 states the same threshold-support condition as a fixed-point specification, denoted SPIN (Algorithm 3); the specification is what Lemma 5.6 appeals to when certifying SPIN*'s correctness.

5.1 Vertex updates from static clique counts

The static-CPI invariant counts s -cliques without rebuilding paths; for core values we additionally need a rule for lowering a vertex value from the cliques that remain. At level k , the vertex needs at least k encoded s -cliques whose other vertices can also sustain level k . For a tentative value vector, define the one-vertex update

$$\Phi_h(v) = \max\left\{k \geq 0 \mid \sum_{P \ni v} W_v(P, h, k) \geq k\right\}.$$

Here $W_v(P, h, k)$ is the number of encoded s -cliques on path P through v whose other vertices have value at least k , and the sum runs over all paths through v . For clique size two, the supporting cliques are edges, so the update becomes the standard core fixed-point test over neighbors [21, 31]. For larger clique sizes, the counted items are s -cliques, not distinct neighbor vertices. The next lemma counts $\sum_{P \ni v} W_v(P, h, k)$ from CPI paths without enumerating those cliques.

Lemma 5.1 (Per-path Witness Count). *Fix a vertex v , a current vector h , and a threshold $k \geq 1$. For a path P with $v \in V_h(P) \cup V_p(P)$, let $H_k(P)$ and $Q_k(P)$ be the number of vertices in $V_h(P) \setminus \{v\}$ and $V_p(P) \setminus \{v\}$, respectively, whose current h -value is at least k . The number $W_v(P, h, k)$ of encoded s -cliques through v in which every other vertex has h -value at least k is:*

- (1) if $H_k(P) < |V_h(P) \setminus \{v\}|$, then $W_v(P, h, k) = 0$;
- (2) if $v \in V_h(P)$, then $W_v(P, h, k) = \binom{Q_k(P)}{\eta_p}$;

Algorithm 2: VertexSupport(v, h): one-vertex update from static clique counts.

Input: Vertex $v \in V$; value array h with $h[u]$ the running estimate of $\kappa_s(u)$ for each $u \in V$; CPI $\mathcal{P}$.
Output: The threshold-support value $\Phi_h(v)$.

```

1  $C[0, \dots, h[v]] \leftarrow 0$ ; // threshold clique counts
2 foreach path  $P \in \mathcal{P}$  through  $v$  do
3   for  $k \leftarrow 1$  to  $h[v]$  do
4      $C[k] \leftarrow C[k] + W_v(P, h, k)$ ; // Lemma 5.1
5 for  $k \leftarrow h[v]$  to 1 do
6   if  $C[k] \geq k$  then return  $k$ ;
7 return 0

```

(3) if $v \in V_p(P)$, then $W_v(P, h, k) = \binom{Q_k(P)}{\eta_p - 1}$.

Summing over the paths through v gives the total threshold count $\sum_{P \ni v} W_v(P, h, k)$, and $\Phi_h(v)$ is the largest k for which this sum is at least k .

PROOF. Each encoded s -clique through v has the form $V_h(P) \cup P'$ with $P' \subseteq V_p(P)$ and $|P'| = \eta_p$. If any other hold vertex fails threshold k , no such clique qualifies; otherwise, $v \in V_h(P)$ leaves all η_p pivot slots to be filled from the $Q_k(P)$ passing pivots and gives $\binom{Q_k(P)}{\eta_p}$ qualifying cliques, while $v \in V_p(P)$ leaves $\eta_p - 1$ slots to fill and gives $\binom{Q_k(P)}{\eta_p - 1}$. Summing over the paths through v gives the stated total. $\square$

VertexSupport(v, h) computes this update for one vertex. During one call, the algorithm builds only local threshold state for v . After any prefix of the paths through v , that local state contains exactly the cliques contributed by the scanned paths. Scanning the next path adds the binomial counts from Lemma 5.1, and the largest threshold with enough cliques is the updated value. The call reads the static CPI but never mutates a path, so the same update rule can be used by both solvers below.

Algorithm 2 zeroes the local threshold-count array up to $h[v]$ (Line 1), accumulates the per-threshold count $W_v(P, h, k)$ into $C[k]$ for every path through v using the closed form of Lemma 5.1 (Lines 2-4), and returns the largest k at which $C[k] \geq k$ by scanning C from $h[v]$ downward (Lines 5-7), or 0 if none. The pseudocode is shown in semantic form for clarity; the implementation never materialises a dense $C[0, \dots, h[v]]$ array (infeasible when $h[v]$ approaches $\deg^{(s)}(v)$, easily $10^{10}+$ on dense graphs) and instead replaces the inner k -loop with a single pass per scanned path that sorts the path's vertex values once and computes $W_v(P, h, \cdot)$ at every threshold via prefix sums. Let $w_v = \sum_{P \ni v} (|V_h(P)| + |V_p(P)|)$ denote the total incidence weight at v , and $L_{\max} = \max_P (|V_h(P)| + |V_p(P)|)$ the maximum path size; per-call cost is $O(w_v \log L_{\max})$. The call reads the static CPI and the vector h , and it mutates no path.

5.2 Fixed-Point Specification

We state the threshold-support condition as a fixed-point specification on the value vector, denoted SPIN: starting from the initial s -clique support, repeatedly apply VertexSupport to every vertex until no entry changes. The only mutable data are the current and next value vectors; the initial vector dominates the final core value pointwise, and each pass can only lower entries. A fixed point means every vertex value is consistent with the cliques whose other vertices have high enough current value, which makes the

Table 2: SPIN trace on the running example. An *interaction* is one full-graph h -update; the *core value* $\kappa_3(v)$ (bold row) is $h^{(t)}[v]$ at the fixed point.

interaction t	h -value $h^{(t)}[v]$ per vertex									
	v_1	v_2	v_3	v_4	v_5	v_6	v_7	v_8	v_9	v_{10}
0 (initial $\deg^{(s)}$)	3	4	6	6	3	4	1	1	1	1
1	3	3	3	3	3	3	1	1	1	1
2 (fixed point = κ_3)	3	3	3	3	3	3	1	1	1	1

Algorithm 3: SPIN: direct static-path fixed-point iteration.

Input: Graph G , clique size s , CPI $\mathcal{P}$.
Output: $\kappa_s(v)$ for every $v \in V$.

```

1 foreach  $v \in V$  do
2    $h[v] \leftarrow \deg^{(s)}(v);$            // closed form from Lemma 2.6
3  $\text{changed} \leftarrow \text{true};$ 
4 while  $\text{changed}$  do
5    $\text{changed} \leftarrow \text{false};$   $h_{\text{new}} \leftarrow h;$ 
6   foreach  $v \in V$  in a fixed order do
7      $h_{\text{new}}[v] \leftarrow \text{VertexSupport}(v, h);$ 
8     if  $h_{\text{new}}[v] \neq h[v]$  then  $\text{changed} \leftarrow \text{true};$ 
9    $h \leftarrow h_{\text{new}};$ 
10 foreach  $v \in V$  do
11    $\kappa_s(v) \leftarrow h[v];$ 
12 return  $\kappa_s$ 
```

converged vector the canonical semantic anchor for any static-CPI core-value solver.

Algorithm 3 initialises $h[v]$ to $\deg^{(s)}(v)$ for every vertex via the closed form of Lemma 2.6 (Lines 1-2). The main loop (Lines 3-9) iterates until no entry changes in a full pass: it sets a *changed* flag to false, copies the current vector into h_{new} (Line 5), and then calls $\text{VertexSupport}(v, h)$ for every vertex in a fixed order and stores the result into $h_{\text{new}}[v]$ (Lines 6-7); if any update lowered an entry the flag is set, otherwise the pass exits. The swap $h \leftarrow h_{\text{new}}$ at the end of the pass (Line 9) avoids order dependence inside the pass. The final core values are read off the converged vector (Lines 10-11) and returned (Line 12); a pass with no changed entry is the certificate that the fixed point has been reached.

Example 5.2 (Static-path update on the running example). Initialise $h[v] \leftarrow \deg^{(s)}(v)$ using Lemma 2.6; on the running example $h[v_4] = 6$ (contributions $2+1+2$ from P_3, P_4, P_5 as pivot and 1 from P_6 as hold; see Table 1). The first call $\text{VertexSupport}(v_4, h)$ scans the four paths through v_4 and counts qualifying cliques per threshold via Lemma 5.1. At threshold $k=3$, every other vertex on each of the four paths has $h \geq 3$, so the count sums are $2+1+2+1 = 6$; the largest k with $\sum_{P \ni v_4} W_{v_4}(P, h, k) \geq k$ is $\Phi_h(v_4) = 3$, the final $\kappa_3(v_4)$. Table 2 shows the full h -value trajectory: interaction 0 carries the initial $\deg^{(s)}$, interaction 1 drops the K_4 vertices $v_1, \dots, v_6$ to 3, and interaction 2 reproduces interaction 1 to certify the fixed point. The converged vector $h^* = \kappa_s$ is the semantic target that any static-CPI core-value algorithm must reach; Section 5.3 delivers SPIN*, which reaches that target in a single deletion pass.

We next establish that this iteration terminates and that the fixed point it reaches is the $(1, s)$ -nucleus core number of Definition 2.4.

Lemma 5.3 (Fixed Point Correctness). *Initialised at $h^{(0)}[v] = \deg^{(s)}(v)$, Algorithm 3 converges in at most $n + 1$ passes to $h[v] = \kappa_s(v)$ for every $v \in V$, and the sequence $h^{(t)}[v]$ is monotonically non-increasing.*

PROOF. By the operator definition, $h^{(t+1)}[v] \geq k$ iff v has at least k incident s -cliques whose other vertices all satisfy $h^{(t)} \geq k$, so $\deg^{(s)}(v)$ is the largest possible value and the sequence is monotone non-increasing on the non-negative integers. Once a vertex's value equals its final $\kappa_s(v)$, the operator never lowers it further (by the converse direction below); each pass that is not already a fixed point lowers the value of at least one not-yet-finalised vertex, so at most n such passes occur before the fixed point h^* is reached and one more pass certifies it. At h^* , let $S_k = \{v \mid h^*[v] \geq k\}$; every $v \in S_k$ has at least k incident s -cliques entirely inside S_k , and each such clique lies in v 's s -connected component $C \subseteq S_k$ (the clique itself provides the connection), so C satisfies Conditions (i)–(iii) of Definition 2.3 and is a k – $(1, s)$ -nucleus, giving $\kappa_s(v) \geq k$. Conversely, if v lies in a k – $(1, s)$ -nucleus C' , we show by induction on t that $h^{(t)}[u] \geq k$ for every $u \in C'$. The base case $t=0$ holds since each $u \in C'$ is in at least k s -cliques inside C' (Definition 2.3), so $h^{(0)}[u] = \deg^{(s)}(u) \geq k$. For the inductive step, assume $h^{(t)}[u'] \geq k$ for every $u' \in C'$; then u has at least k s -cliques inside C' whose other vertices all satisfy $h^{(t)} \geq k$, so the operator yields $h^{(t+1)}[u] \geq k$. Therefore $h^*[v] \geq k$, and hence $h^*[v] = \kappa_s(v)$. $\square$ $\square$

The cost is the per-pass work (one VertexSupport call per vertex) times the number of passes:

Theorem 5.4 (SPIN Complexity). *Algorithm 3 computes κ_s in $O(T \cdot \Sigma \cdot L_{\max} \cdot \log L_{\max})$ work, where Σ is the static CPI's total vertex–path incidence count (Section 2), $T \leq n + 1$ is the number of passes (Lemma 5.3), and $L_{\max}$ is the maximum path size (Section 5.1).*

PROOF. Each pass calls $\text{VertexSupport}(v, h)$ for every $v \in V$ at per-call cost $O(w_v \log L_{\max})$ (Section 5.1). Summing, $\sum_v w_v = \sum_P |P|^2 \leq L_{\max} \cdot \Sigma$, so a pass costs $O(\Sigma \cdot L_{\max} \cdot \log L_{\max})$, and Lemma 5.3 bounds the number of passes by $n + 1$. $\square$ $\square$

5.3 Deletion-Order Peel Solver

SPIN* is a single-pass bucket-queue peel solver over the static CPI. For each path, it maintains a liveness bit, an integer pivot counter (the number of pivot vertices not yet finalised), and the fixed pivot requirement. For each unfinalised vertex, it maintains a bucket-queue key equal to that vertex's current threshold-support value over the live path counters. Each iteration pops a minimum-key vertex, assigns it its peel level as core value, and walks the paths incident to it: hold removals retire the path; pivot removals decrement the counter and retire the path once $p_P < \eta_P$ (Theorem 4.1). Path-mates whose support drops are re-keyed via the binomial loss formula of Lemma 4.2 and moved to a lower bucket; the peel terminates after n pops, and the assigned $\kappa_s(v)$ is final at the moment of the pop.

Algorithm 4 outputs the core values $\kappa_s(v)$ for every $v \in V$, and emits a sorted deletion log as a byproduct. Its running time depends on U , the number of incremental support-update events triggered along the peel.

Algorithm 4 first initialises every path's pivot count p_P , pivot requirement η_P , and liveness bit (Lines 1-2), then sets each vertex's working value $h[v]$ to its initial s -clique support and inserts v into the corresponding bucket (Lines 3-4). The main loop (Lines 6-19) advances the peel level k to the smallest non-empty bucket and pops the next vertex v (Line 7), finalises $\kappa_s(v) \leftarrow k$ (Line 8), and walks

Algorithm 4: SPIN*: core values via single-pass bucket-queue peeling.

Input: Graph G , clique size s , CPI $\mathcal{P}$.
Output: $\kappa_s(v)$ for every $v \in V$.

```

1 foreach path  $P \in \mathcal{P}$  do
2    $p_P \leftarrow |V_P(P)|$ ;  $\eta_P \leftarrow s - |V_h(P)|$ ;  $\text{alive}_P \leftarrow (p_P \geq \eta_P)$ ;
3 foreach  $v \in V$  do
4    $h[v] \leftarrow \deg^{(s)}(v)$ ; insert  $v$  into bucket  $B[h[v]]$ ;
5  $k \leftarrow 0$ ;  $V_{\text{fin}} \leftarrow \emptyset$ ;
6 while  $|V_{\text{fin}}| < n$  do
7   advance  $k$  until  $B[k] \neq \emptyset$ ; pop  $v$  from  $B[k]$ ;
8    $\kappa_s(v) \leftarrow k$ ; append  $(v, k)$  to the deletion log;  $V_{\text{fin}} \leftarrow V_{\text{fin}} \cup \{v\}$ ;
9    $\text{Aff} \leftarrow \emptyset$ ;
10  foreach  $P$  with  $\text{alive}_P$  and  $v \in V_h(P) \cup V_p(P)$  do
11     $\text{Aff} \leftarrow \text{Aff} \cup ((V_h(P) \cup V_p(P)) \setminus \{v\})$ ;
12    if  $v \in V_h(P)$  then
13       $\text{alive}_P \leftarrow \text{false}$ ; // path death
14    else
15       $p_P \leftarrow p_P - 1$ ;
16      if  $p_P < \eta_P$  then  $\text{alive}_P \leftarrow \text{false}$ ;
17  foreach  $u \in \text{Aff} \setminus V_{\text{fin}}$  do
18     $y \leftarrow \max\{k, \text{VertexSupport}(u, h)\}$ ; // implemented
19    incrementally with Lemma 4.2
20    if  $y < h[u]$  then move  $u$  from  $B[h[u]]$  to  $B[y]$ ;  $h[u] \leftarrow y$ ;
21 return  $\kappa_s$ 

```

every live path that contains v (Lines 10-16): if v is a hold, the path dies (Line 13); if v is a pivot, the path's pivot counter decrements, and the path also dies once $p_P < \eta_P$ (Lines 15-16). Affected path-mates that have not been finalised are recomputed via Lemma 4.2 (Line 18) and moved to a lower bucket if their value dropped (Line 19). The pseudocode writes the affected update as `VertexSupport` to keep the fixed-point semantics visible; the implementation evaluates the changed contribution with the binomial loss formula of Lemma 4.2 rather than recomputing from scratch, so the total work scales with the number of incremental update events U . Buckets maintain the minimum- h vertex because values only decrease and are bounded by $\kappa_{\max}$ [5].

Example 5.5 (Peel iteration on the running example). Initialise $h[v] \leftarrow \deg^{(s)}(v)$ as in Example 5.2 and place every v in bucket $B[h[v]]$; on the running example, v_7, v_8, v_9, v_{10} sit in $B[1]$ and pop first at peel level $k = 1$, retiring P_1 and P_2 through Case C' of Theorem 4.1. At iteration 7 the cursor has advanced to $k=3$ and v_4 pops; the algorithm walks the two live paths through v_4 (Table 1). $P_5 = (\{v_3\}, \{v_4, v_5, v_6\})$ has v_4 as a pivot, so p_{P_5} shrinks $3 \rightarrow 2$ (Case C; by Lemma 4.2, v_3 loses $\binom{3}{2} - \binom{2}{2} = 2$ from its bucket key and v_5, v_6 each lose $\binom{2}{1} - \binom{1}{1} = 1$); $P_6 = (\{v_4\}, \{v_5, v_6\})$ has v_4 as a hold and dies (Case A; surviving pivots v_5, v_6 each lose $\binom{1}{1} = 1$). After ten pops the peel completes with $\kappa_3(v_i) = 3$ for $i \in \{1, \dots, 6\}$ and $\kappa_3(v_i) = 1$ for $i \in \{7, \dots, 10\}$; Table 3 shows every vertex's h -value across all ten interactions.

The converged vector is the fixed point of the specification in Section 5.2:

Lemma 5.6 (SPIN* Equivalence). *Algorithm 4 returns the same κ_s values as Algorithm 3.*

PROOF. Both algorithms apply the same per-vertex threshold-support condition; the difference is only the schedule. SPIN recomputes the condition by full passes, while SPIN* uses Theorem 4.1 and Lemma 4.2 to update exactly the path contributions changed

Table 3: SPIN* trace on the running example. An *interaction* is one deletion event; the bold entry in row t , column v is the *core value* $\kappa_3(v)$ assigned when v is popped.

interaction t	h -value $h^{(t)}[v]$ per vertex (bold = $\kappa_3(v)$)									
	v_1	v_2	v_3	v_4	v_5	v_6	v_7	v_8	v_9	v_{10}
0 (init)	3	4	6	6	3	4	1	1	1	1
1	3	4	6	6	3	3	0	1	1	1
2	3	4	6	6	3	3	1		1	1
3	3	3	6	6	3	3			0	1
4	3	3	6	6	3	3			1	
5	1	3	4	4	3	3				
6	3		3	3	3	3				
7			1	3	1	1				
8			0		3	0				
9			3			0				
10						3				

by the popped vertex. The path counters of SPIN* are support-equivalent to physically deleting finalised vertices from the CPI, so the bucket key of every unfinalised vertex is its residual threshold-support value clipped from below by the current peel level. Initial values $h[u] = \deg^{(s)}(u)$ are upper bounds and every update only decreases a key, so no vertex is ever assigned a value larger than its current residual threshold support above the current peel level. Consider the moment when the bucket cursor first reaches level k : the remaining vertices with key at least k form a super-level set in which each vertex has at least k qualifying s -cliques, so each s -connected component of that set is a k -(1, s)-nucleus and every vertex later popped while the cursor stays at level k has core value at least k . When a vertex v is popped from bucket $B[k]$, it cannot belong to any $(k+1)$ -(1, s)-nucleus, since otherwise the exact residual threshold-support count at threshold $k+1$ would keep its key at least $k+1$ until it was removed. Thus the pop assigns $\kappa_s(v) = k$, and by Lemma 5.3, κ_s is also the fixed point reached by SPIN. $\square$ $\square$

The runtime decomposes into the initialisation, the path-event walk on each peel, and the incremental support-refresh events the walk triggers:

Theorem 5.7 (SPIN* Complexity). *Algorithm 4 runs in $O(\Sigma + U + n)$ time, where Σ is the total vertex-path incidence count of the static CPI (Section 2) and U is the total number of incremental support-refresh events over all affected path-mates.*

PROOF. Initialization of vertex values and path counters costs $O(n + \Sigma)$; each vertex-path incidence is visited when its vertex is finalized, so path-event processing costs $O(\Sigma)$; each affected incremental refresh contributes one unit to U ; bucket moves and bucket pops are $O(1)$ amortized; and the bucket cursor advances through at most n distinct values (one key per vertex at any time). The total time is therefore $O(\Sigma + U + n)$. $\square$ $\square$

The mutable baseline pays heap and residual-index maintenance costs on the same logical peel events; Table 3 traces SPIN* on the running example, with each row one deletion interaction t , each column one vertex, and the bold entry in row t , column v the $\kappa_3(v)$ assigned when v is popped (cells past that pop in the same column are blank because v no longer participates). The experiments in Section 8 measure the impact of replacing those operations with static path counters.

6 Parallel CPI Construction

Once SPIN* makes peeling counter-only, CPI construction becomes the load-bearing cost of the pipeline (Figure 3). This bottleneck

Algorithm 5: ParaBuild: parallel CPI construction.

Input: Graph $G = (V, E)$; clique size s ; thread count T .
Output: Static CPI $\mathcal{P}$.

- 1 Compute a degeneracy ordering $(v_1, \dots, v_n)$ of G and orient every edge from lower to higher rank, giving $N^+(\cdot)$;
- 2 Partition the seed sequence $(v_1, \dots, v_n)$ into T contiguous chunks $C_1, \dots, C_T$;
- 3 **foreach** thread $t \in \{1, \dots, T\}$ in parallel **do**
- 4 $\mathcal{P}_t \leftarrow \emptyset$;
- 5 **foreach** seed vertex $v \in C_t$ **do**
- 6 $\text{EXPANDPATH}_t(N^+(v), \{v\}, \emptyset)$; // one root per vertex
- 7 $\mathcal{P} \leftarrow \bigcup_{t=1}^T \mathcal{P}_t$;
- 8 **return** $\mathcal{P}$ **Function** $\text{ExpandPath}_t(P, V_h, V_p)$:
- 9 **if** $|V_h| > s$ **then return**;
- 10 **if** $P = \emptyset$ **then**
- 11 **if** $|V_h| + |V_p| \geq s$ **then** append the path (V_h, V_p) to $\mathcal{P}_t$;
- 12 **return**
- 13 $u_{\text{piv}} \leftarrow \arg \max_{x \in P} |P \cap N^+(x)|$; // Tomita pivot
- 14 **foreach** $v \in P \setminus N(u_{\text{piv}})$ **do**
- 15 $P \leftarrow P \setminus \{v\}$;
- 16 **if** $v = u_{\text{piv}}$ **then**
- 17 $\text{ExpandPath}_t(P \cap N(v), V_h, V_p \cup \{v\})$;
- 18 **else**
- 19 $\text{ExpandPath}_t(P \cap N(v), V_h \cup \{v\}, V_p)$;

shift is why construction is part of the algorithmic pipeline rather than a detached implementation detail. We observe that the static-CPI invariant makes construction embarrassingly parallel. With no mutation contract crossing threads and each s -clique uniquely owned by its smallest-rank seed, the standard degeneracy-ordered Bron–Kerbosch template runs lock-free across T threads.

Why the seed loop can be parallel. The CPI builder follows the standard degeneracy-ordered Bron–Kerbosch search with Tomita pivoting [9, 15, 44, 49]. Each s -clique has one owner: the smallest vertex of the clique in the degeneracy order. Therefore, the outer loop over seed vertices partitions the emitted s -clique families. No two seeds own the same s -clique.

Static CPI also changes what construction must produce. The mutable baseline builds residual tree state because later peeling may edit that state. Our $r = 1$ pipeline never edits the stored hold or pivot sets. The builder therefore only has to emit the path set $\mathcal{P}$; no per-path mutation contract has to be maintained across the parallel boundary.

Independent per-thread enumeration. ParaBuild takes G , s , and a thread count T . Each thread owns a contiguous range of seed vertices and runs the same seed-rooted enumeration as the sequential builder on each of its seeds, accumulating the emitted paths into its own output set $\mathcal{P}_t$. The degeneracy ordering gives every s -clique a unique smallest-rank owner, so the sets $\{\mathcal{P}_t\}_t$ are pairwise disjoint; the parallel output is therefore the disjoint union $\mathcal{P} = \bigcup_t \mathcal{P}_t$, and the merge is purely the set union, with no synchronisation across threads during enumeration and no deduplication or rewrite after. The expensive work is the Bron–Kerbosch search under each seed; the merge contributes no per-path arithmetic.

Algorithm 5 mirrors Algorithm 1 line-for-line: Line 1 computes the same degeneracy ordering and edge orientation, and the function body EXPANDPATH_t at the bottom is identical to Algorithm 1’s EXPANDPATH except that it appends to the thread-local output $\mathcal{P}_t$ instead of the global $\mathcal{P}$. The only structural change is at the driver loop: Algorithm 1’s sequential *for over seeds* is replaced by Line 2

partitioning the seed sequence into T chunks and Lines 3–7 running one chunk per thread, with Line 8 merging the thread-local outputs. Every s -clique has a unique smallest-rank owner under the degeneracy ordering, so the chunks $C_1, \dots, C_T$ induce a partition of $\mathcal{P}$: each path is emitted by exactly one thread, so the per-thread outputs are pairwise disjoint and $\bigcup_t \mathcal{P}_t$ is the same set of paths Algorithm 1 would have produced sequentially. The choice of T , the partition of seeds, and the in-memory layout used to store $\mathcal{P}$ are implementation knobs; they affect throughput but not the output set.

Correctness. ParaBuild is a parallel schedule of the same seed-owned enumeration. Seed ownership gives disjoint emitted s -clique families. The deterministic merge changes only the storage order, not the represented path families. The resulting path set satisfies Definition 2.5. Therefore, the support formula of Lemma 2.6 applies unchanged.

Scope. The design parallelizes construction because construction is the phase exposed by the static-CPI pipeline. It does not claim a lower bound showing that SPIN* cannot be parallelized. It also does not claim ideal scaling. The scaling evidence is reported in Figure 4.

7 Core-Value Hierarchy Construction

Because the static-CPI invariant of Section 4 keeps every leaf intact through peeling, the hierarchy is no longer a separate decomposition problem. After the core values are known, every CPI leaf encodes the same family of s -cliques both before and after the peel; the elder-rule join tree of Definition 2.3 can therefore be emitted in $O(\Sigma \log \Sigma)$ time without re-enumerating a single clique, by generating a short list of merge events per leaf that tracks the levels at which its encoded s -cliques become alive and then scanning those events in descending level with a union-find. Algorithms that mutate the CPI during peeling cannot reuse it for this purpose: they must either re-run s -clique enumeration at each level or maintain one parallel disjoint-set forest per core level during peeling, which blows up memory by an order of magnitude (Figure 7). For any level k , the super-level set is $V_k = \{v \mid \kappa_s(v) \geq k\}$, and Definition 2.3 measures connectivity inside V_k through s -cliques contained in V_k ; BuildHier (Algorithm 6) materialises that filtration from the preserved leaves.

Per-clique events from each leaf. A CPI leaf P encodes the family $\{V_h(P) \cup X \mid X \subseteq V_p(P), |X| = \eta_p\}$ of s -cliques (Definition 2.5). A given encoded clique $V_h(P) \cup X$ is alive in V_k iff $\min_{v \in V_h(P) \cup X} \kappa_s(v) \geq k$, so different cliques of the same leaf may first appear at different levels. For each leaf we therefore generate a short ordered list of *merge events* that exactly track the level at which a new encoded clique enters V_k . Let $h_{\min} = \min_{v \in V_h(P)} \kappa_s(v)$ (or $+\infty$ if $V_h(P) = \emptyset$) and sort $V_p(P)$ as $p_1, \dots, p_m$ with $\kappa_s(p_1) \geq \dots \geq \kappa_s(p_m)$. When $\eta_p = 0$ the leaf encodes the single clique $V_h(P)$, fired at level $h_{\min}$ with member set $V_h(P)$. When $\eta_p \geq 1$, the first event fires at level $k^{(\eta_p)} = \min(h_{\min}, \kappa_s(p_{\eta_p}))$ and unifies $V_h(P) \cup \{p_1, \dots, p_{\eta_p}\}$, the highest-birth encoded clique. For each $i = \eta_p + 1, \dots, m$ a follow-up event fires at level $k^{(i)} = \min(h_{\min}, \kappa_s(p_i))$ and unifies the new pivot p_i into the leaf’s already-formed component (concretely with p_1 as anchor), because adding p_i activates the encoded cliques that contain p_i together with $\eta_p - 1$ already-active pivots. The list of events from a single leaf has length $|V_p(P)| - \eta_p + 1$

Algorithm 6: BuildHier: elder-rule join tree from a static CPI.

Input: Graph $G = (V, E)$; clique size s .
Output: Elder-rule join tree T of the s -clique-connected super-level filtration.

```

1   $(\mathcal{P}, \kappa_s) \leftarrow \text{SPIN}^*(G, s)$ ; // Algorithm 4: static CPI (preserved) +
   core values
2   $\mathcal{E} \leftarrow []$ ; // merge events  $(k, S)$ : unify the vertex set  $S$  at
   level  $k$ 
3  foreach leaf  $P \in \mathcal{P}$  with  $|V_h(P)| + |V_p(P)| \geq s$  do
4       $h_{\min} \leftarrow \min\{\kappa_s(v) \mid v \in V_h(P)\}$ ; //  $+\infty$  if  $V_h(P) = \emptyset$ 
5      if  $\eta_P = 0$  then
6          append  $(h_{\min}, V_h(P))$  to  $\mathcal{E}$ ;
7      else
8          sort  $V_p(P)$  as  $p_1, \dots, p_m$  with  $\kappa_s(p_1) \geq \dots \geq \kappa_s(p_m)$ ;
9          append  $(\min(h_{\min}, \kappa_s(p_{\eta_P})), V_h(P) \cup \{p_1, \dots, p_{\eta_P}\})$  to  $\mathcal{E}$ ;
           // first event
10         foreach  $i \leftarrow \eta_P + 1$  to  $m$  do
11             append  $(\min(h_{\min}, \kappa_s(p_i)), \{p_1, p_i\})$  to  $\mathcal{E}$ ; //  $p_i$  joins
               via anchor  $p_1$ 
12 sort  $\mathcal{E}$  by nonincreasing level (ties broken by event order);
13 initialise union-find  $U$  over  $V$ ;  $\text{birth}[r] \leftarrow +\infty$  for every root  $r$ ;
14  $T \leftarrow$  empty forest;
15 foreach event  $(k, S)$  in sorted order do
16      $R \leftarrow \{\text{FIND}(x) \mid x \in S\}$ ;
17     foreach root  $r \in R$  do
18         if  $\text{birth}[r] = +\infty$  then
19             emit a hierarchy node  $n_r$  with  $k_{\text{birth}} \leftarrow k$ ;  $\text{birth}[r] \leftarrow k$ ;
20     if  $|R| \geq 2$  then
21         sort  $R$  descending by  $(\text{birth}[r], \text{id}(r))$ ; let  $r^*$  be the elder;
           // elder rule
22         foreach  $r \in R \setminus \{r^*\}$  do
23             attach  $n_r$  as child of  $n_{r^*}$  at level  $k$  in  $T$ ; // younger dies
24         UNION all of  $R$  into a single component;
25 return  $T$ 

```

when $\eta_P \geq 1$, and length 1 when $\eta_P = 0$; in both cases the events fire at nonincreasing levels.

Reverse scan over events. BuildHier concatenates events across all leaves and processes them in nonincreasing level. Each event (k, S) unifies the vertices in S via union-find with the elder rule: the earliest-born component absorbs the others, and each younger component dies into the elder at level k . A vertex first appearing in some event with $k_{\text{event}} \geq 1$ is born at the highest level at which it is unified; vertices that never appear ($\kappa_s(v) = 0$) are omitted from the hierarchy.

Algorithm 6 invokes SPIN* once (Line 1) to obtain the static CPI $\mathcal{P}$ and the core-value vector κ_s , then runs in two phases. Phase 1 (Lines 2–11) generates merge events: for each leaf with at least one encoded s -clique, it sorts the pivot set by κ_s in descending order and emits one event for the highest-birth encoded clique (size $|V_h| + \eta_P$), followed by unit events that each splice one lower- κ pivot into the leaf's already-formed component. Phase 2 (Lines 12–24) processes events in nonincreasing level: for each event (k, S) , the union-find returns the root set R of S , emits a new hierarchy node for any root born here for the first time (Lines 17–19), and applies the elder rule to merge multiple roots into the eldest (Lines 20–24). The returned forest T is the elder-rule join tree of the s -clique-connected super-level filtration of (V, κ_s) .

Correctness. Fix a level k and let C be any s -clique alive in V_k . Uniqueness of the CPI encoding (Definition 2.5) gives the leaf P such that $C = V_h(P) \cup X$ with $X \subseteq V_p(P)$, $|X| = \eta_P$. All members of X are in V_k , so each is among the top $|X| = \eta_P$ pivots of P once $V_p(P)$

is restricted to V_k ; hence at the moment C enters the filtration (level $\min_{v \in C} \kappa_s(v)$), the corresponding event has either already fired (if C uses only the top η_P pivots) or fires at exactly that level (when C uses the new pivot p_i , the event $(k^{(i)}, \{p_1, p_i\})$ joins p_i to the leaf's component). Either way the vertices of C are in one component at level $\min_{v \in C} \kappa_s(v)$, so every Definition 2.3 merge is performed at the correct level. The elder rule preserves the component with earlier birth and attaches the younger one in the join tree [14].

Complexity. Phase 1 (event generation) scans each leaf once and sorts its pivot set in $O(\Sigma + \sum_P |V_p(P)| \log |V_p(P)|) \subseteq O(\Sigma \log L_{\max})$ ($L_{\max}$ = maximum path size, Section 5.1). The number of events $|\mathcal{E}|$ is at most $\sum_P \max(1, |V_p(P)| - \eta_P + 1) \leq \Sigma$, and sorting them by level costs $O(\Sigma \log \Sigma)$. Phase 2 (event processing) performs at most $\sum_P (|V_h(P)| + \eta_P) + \sum_{P: \eta_P \geq 1} 2(|V_p(P)| - \eta_P) \leq 2\Sigma$ union-find operations in total (the first event of each leaf unifies $|V_h(P)| + \eta_P$ elements; for $\eta_P \geq 1$ leaves each of the $|V_p(P)| - \eta_P$ follow-up events unifies a pair), at $O(\Sigma \alpha(n + |\mathcal{P}|))$ cost over the DSU's $n + |\mathcal{P}|$ elements. Total time is $O(\Sigma \log \Sigma)$ with no clique re-enumeration: BuildHier touches each CPI incidence a constant number of times. **Memory.** Beyond the union-find of size $n + |\mathcal{P}|$ (the implementation uses each CPI leaf as an auxiliary connector node, so that one union per event suffices to bind every member of S), the routine needs a list of vertices per leaf to fire unions. SPIN*'s vertex-to-path inverse CSR (Section 5) supports a single bucket-CSR pass that materialises the per-leaf lists in $O(\Sigma)$ temporary memory, freed on return. The static CPI budget of Section 8 therefore holds throughout peeling proper and is only briefly elevated during the hierarchy emit.

Consequence. The clique-coherent core-value hierarchy is not a separate decomposition problem in this pipeline: it is a byproduct of the static CPI surviving peeling. Each preserved leaf contributes a short sequence of merge events tracking when its encoded cliques become alive, and a single descending union-find pass over those events produces the join tree. Methods that mutate the CPI during peeling cannot reuse it for hierarchy emit and need a second clique-enumeration pass [39].

Section 8 (Exp-7) compares BuildHier against the only feasible baseline at $r=1$, namely CND's general (r, s) level-DSU hierarchy specialised to $r=1$, and confirms that BuildHier uses up to $43\times$ less memory (gmean $13\times$ over 24 matched cells) and up to $35\times$ less hierarchy-emission wall time (gmean $7.7\times$ over the same cells), with the gap growing at larger s where the encoded-clique mass dominates.

8 Experiments

Exactness follows from the invariants and fixed-point equivalence proved earlier; the experiments measure cost. We report (Exp-1) end-to-end time and memory for SPIN*, (Exp-2) build-vs-peel phase breakdown, (Exp-3) parallel build scaling, (Exp-4) SPIN as the direct fixed-point baseline, (Exp-5) billion-edge com-friendster, (Exp-6) vertex-induced subsampling scalability, and (Exp-7) hierarchy emission against CND's level-DSU baseline.

Hardware and software. All experiments run on an Intel Xeon Gold 6248R server with 503 GB DDR4 RAM under Ubuntu 22.04. The code¹ is compiled with Clang 18.1 (-O3 -march=native) and

¹Code, bench harness, and result CSVs will be released under an anonymised archive at submission.

Table 4: Datasets.

Graph	V	E	avg. deg.
com-amazon [48]	334,863	925,872	5.5
com-dblp [48]	317,080	1,049,866	6.6
twitter [48]	81,306	1,342,296	33.0
web-Stanford [48]	281,903	1,992,636	14.1
com-youtube [48]	1,134,890	2,987,624	5.3
web-Google [48]	875,713	4,322,051	9.9
wiki-Talk [48]	2,394,385	4,659,565	3.9
web-it-2004 [48]	509,338	7,178,413	28.2
soc-pokec [48]	1,632,803	22,301,964	27.3
com-orkut [48]	3,072,441	117,185,083	76.3
com-Friendster [48]	65,608,366	1,806,067,135	55.1

Table 5: Per-incidence memory budget at $\Sigma \gg n$.

Component	Baseline	SPIN*
BK recursion tree T	8Σ	0
Path-to-vertex storage (V_h, V_p)	0	4Σ
Per-path vertex set H_p	32Σ	
Reverse vertex-to-path map	48Σ	
Vertex-to-path incidence		4Σ
Support array	$4n$	$4n$
Heap / bucket array	$24n$	$16n + 8B$
Analytical Σ-coefficient	$\approx 88\Sigma$	$\approx 8\Sigma$
Measured: com-youtube, $s=5$	642 MB	387 MB
Measured: web-it-2004, $s=3$	556 MB	270 MB
Measured: web-Stanford, $s=10$	261 MB	146 MB

linked against TBB 2021.10, Boost 1.83, and robin_hood. Single-threaded measurements use one core; parallel runs use the thread counts in Figure 4.

Datasets. Table 4 lists the benchmark graphs (collaboration, e-commerce, social, web). The sweep on the ten smaller graphs yields 451 matched cells where both SPIN* and CND complete; the headline speedup numbers are over this matched subset. com-friendster is run standalone (Section 8) because CND cannot complete on it. **Algorithms and methodology.** CND [49] is the mutable-CPI baseline (Section 3) restricted to $r=1$; SPIN* is this paper’s static-CPI pipeline; SPIN is the direct fixed-point reference (Section 5.2). All three consume the same edge list on the same hardware, so the comparison isolates mutability vs. static-state peeling. For each (graph, s) cell we record end-to-end time (load + degeneracy + CPI build + peel + output) and peak memory.

Exp-1: peel time and memory. Across the ten graphs (Figure 2), SPIN* sits below CND on every peel-time cell, with a peel-time speedup of $\text{gmean } 17.90\times / \text{max } 49.48\times$ and a memory reduction of $\text{gmean } 1.55\times / \text{max } 3.92\times$. The end-to-end (wall-clock) gap is smaller ($\text{gmean } 1.05\times$) because the shared CPI construction phase dominates once peeling is cheap; this bottleneck shift motivates ParaBuild (Section 6). SPIN is also dominated by SPIN* on all but the easiest cells; the full comparison is in Exp-4 below.

The memory improvement is explained by the incidence-level budget in Table 5. The mutable baseline stores path hash sets and reverse hash maps. SPIN* stores flat path-to-vertex and vertex-to-path arrays plus path counters. The analytical per-incidence gap is larger than the observed memory gap because graph adjacency, allocator overhead, and per-vertex structures do not scale with Σ .

Exp-2: phase breakdown. Sweeping $s \in \{3, 5, 7, 9, 11, 13, 15\}$ on six graphs (Figure 3), the peel slice shrinks with s : it falls below 1% of total time by $s=11$ on medium and large graphs, and stays under 0.1% on web-it-2004 throughout. The bottleneck has shifted onto construction, which is why ParaBuild is part of the algorithmic pipeline.

Exp-3: parallel scaling of CPI construction. ParaBuild streams each seed’s emitted paths into thread-local storage. Figure 4 reports build time on six graphs as T grows from 1 to 64 ($s=3$, median of

three runs): the optimum lies at $T=24$ –48 on dense large graphs and at $T=16$ on smaller or sparser inputs, with a headline single-cell speedup of $23.4\times$ at $T=24$ on web-it-2004 $s=3$ ($92.3\text{ s} \rightarrow 3.95\text{ s}$).

Exp-4: SPIN* vs. SPIN on the same fixed point. SPIN (Algorithm 3) iterates the fixed point by full passes; SPIN* (Algorithm 4) reaches the same point by deletion-ordered events. Both read the same static index (Lemma 5.6), so the comparison isolates the schedule cost. Across the 272 matched cells, SPIN* delivers a $30.4\times$ geometric-mean peel-time speedup, $\text{max } 2,369\times$ on web-it-2004 at $s=53$; the worst case is $0.62\times$ on easy cells where SPIN’s tight loop converges in one or two passes. SPIN hits the 1h cap on com-orkut ($s \geq 3$) and wiki-Talk ($s=10$); SPIN* finishes both in seconds to minutes. The memory profiles are essentially identical ($\text{gmean ratio } 0.97$ over 291 memory-paired cells): the bucket queue and per-leaf counter cache that SPIN* adds amount to a sub-percent premium against the shared static index, a textbook “space for time”. The paper carries both algorithms because they serve different roles: SPIN as the direct semantic formulation, SPIN* as its event-driven realisation.

Exp-5: scaling to a billion-edge graph. To push an order of magnitude past com-orkut (117M edges), we run SPIN* with $T=24$ on com-friendster (65M vertices, 1.806B edges; the largest SNAP graph). SPIN* completes at every tested $s \in \{2, 3, 4, 5, 6, 11\}$. CND OOM-kills (exceeds the 503 GB budget during CPI construction) at every $s \geq 3$ (Figure 5). On the matched cell $s=2$, SPIN* peels in 344 s vs. CND’s 1,775 s ($5.2\times$) using 160 GB vs. 400 GB ($2.5\times$ less); the memory headroom is what lets SPIN* continue past the budget. At higher s the static index stays tractable ($\Sigma \approx 2.26 \times 10^{10}$ at $s=6$, 651 s peel; $\Sigma \approx 1.68 \times 10^{10}$ at $s=11$, 167 s peel). End-to-end runtime is build-dominated (5–7 k s construction vs. 167–676 s peel), confirming that the bottleneck shift carries over to the billion-edge regime and that the parallel builder of Section 6 is what makes the pipeline usable at this scale.

Exp-6: scaling with input size. To probe scaling with the input itself, we vertex-induce subsamples at ratios $\rho \in \{0.2, 0.4, 0.6, 0.8, 1.0\}$ from a fixed-seed random vertex set (nested across ρ); this is the natural choice because SPIN*’s work scales with Σ , while uniform edge removal collapses Σ super-linearly and produces jagged curves. Figure 6 re-runs SPIN* for $s \in \{2, 3, 5, 7, 9, 11, 13, 15\}$ on six graphs spanning two orders of magnitude in edge count (com-dblp, com-youtube, web-Stanford, web-Google, soc-pokec, com-orkut). On every graph the per- ρ curves are cleanly ordered with no inversions on the log scale, so the static-CPI pipeline scales smoothly from sparse collaboration graphs to dense social graphs. com-orkut exposes the natural ceiling at high s : its 100% curve cuts off near $s=5$ because the encoded clique count exceeds the 30-minute budget.

Exp-7: hierarchy emission against the level-DSU baseline. BuildHier (Section 7) consumes the preserved CPI once peeling finishes; the only available $(1, s)$ baseline producing the same hierarchy is CND’s level-DSU emit (specialised to $r=1$, $O(K \cdot \Sigma)$ with K the number of distinct core values). Figure 7 compares four variants on four graphs across $s \in \{3, 5, 7, 9, 11, 13, 15\}$: SPIN*, SPIN*+BuildHier, CND, CND+HIER. BuildHier adds modest overhead on top of SPIN*’s peel state ($\text{gmean memory } 1.26\times$, $\text{max } 2.41\times$; blue and green curves nearly overlap). CND+HIER uses up to $43\times$ more memory ($13\times$ gmean over 24 cells). Even without a hierarchy, CND already uses more memory than SPIN*+BuildHier. On

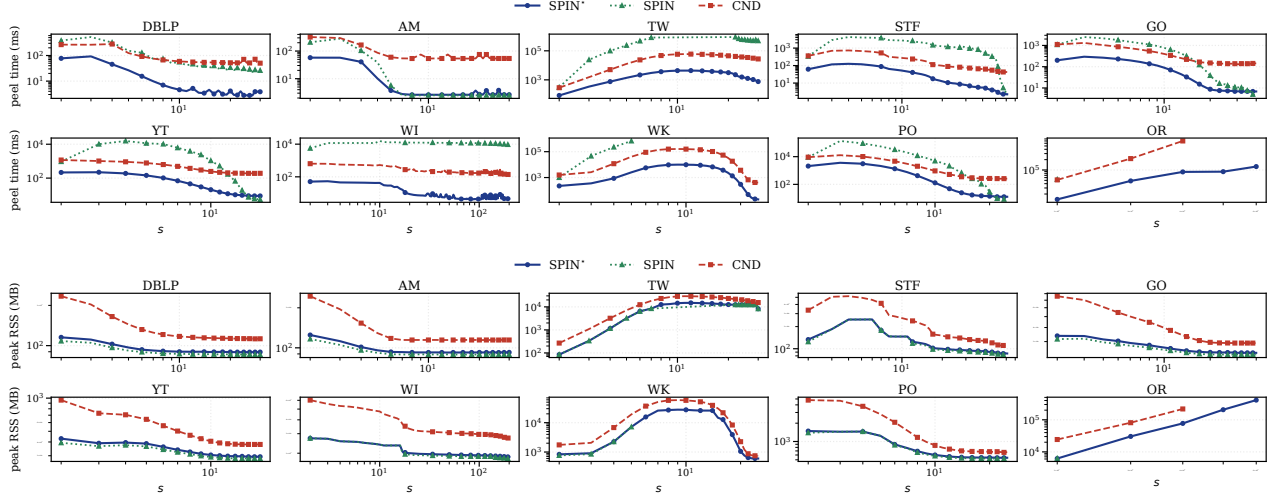


Figure 2: Peel time (top) and peak memory (bottom) across ten graphs for SPIN*, SPIN, and CND.

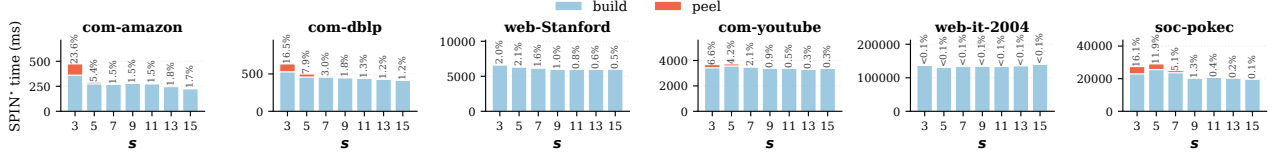
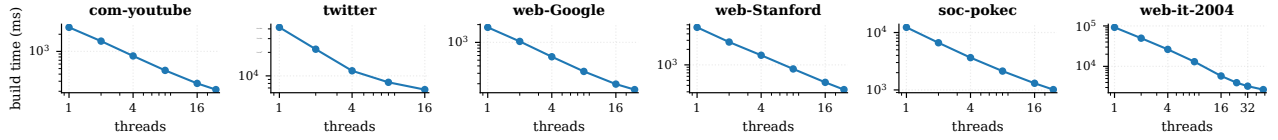
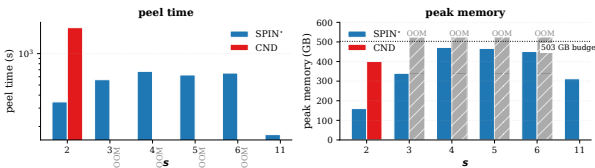


Figure 3: SPIN* phase breakdown on six graphs: build (blue, bottom) vs peel (orange, top). Linear scale; the peel share is printed above each bar.

Figure 4: ParaBuild build time vs. thread count at $s=3$; each curve ends at its optimum T .Figure 5: com-friendster (1.806B edges): SPIN* completes at every tested s ; CND OOMs at $s \geq 3$.

wiki-Talk at $s \geq 9$, both CND variants exceed the 503 GB budget before producing a result, while ours finishes the entire sweep. On time, BuildHier is up to $35\times$ faster than CND+HIER's union work (gmean $7.7\times$), consistent with the $O(\Sigma \log \Sigma)$ vs. $O(K \cdot \Sigma)$ bounds.

Limitations. The static-CPI invariant is specific to $r=1$: for $r \geq 2$, edge- or triangle-level peel events can destroy cliques a path stores, and our pipeline does not subsume the mutable-CPI machinery of [49]. Memory is the binding resource on dense inputs: Σ grows superpolynomially with s , so the largest reachable s is RAM-bounded,

not algorithm-bounded. Support counters $\binom{p}{\eta}$ exceed 2^{63} at moderate p ; the implementation guards the threshold pass with 128-bit accumulators on overflow, but applications needing exact magnitudes must do the same. The experiments are single-machine; distributed deployments and dynamic updates are out of scope.

Reproducibility. Build commands, dataset URLs, and bench drivers are version-controlled with the source; each binary run is wrapped in `/usr/bin/time -v` so peak memory, page faults, and exit status are captured per CSV row.

Takeaway. SPIN* is faster and lighter than CND on the 451 matched cells; the construction phase emerges as the new dominant cost once peeling is counter-only, ParaBuild addresses that shift, and SPIN confirms the direct fixed-point view's role as semantic reference rather than practical solver.

9 Case Studies

The experiments quantify cost; the case studies establish that the $r=1$ specialisation is worth optimising in the first place. On the ground-truth tasks reported here, $(1, s)$ matches the quality of

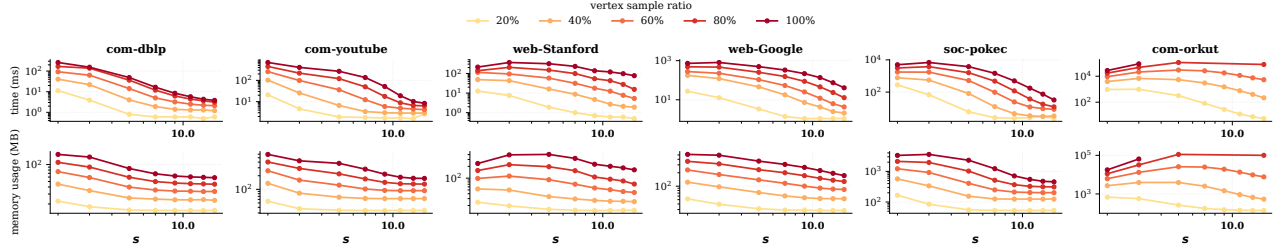


Figure 6: Vertex-induced scalability: time (top) and memory usage (bottom).

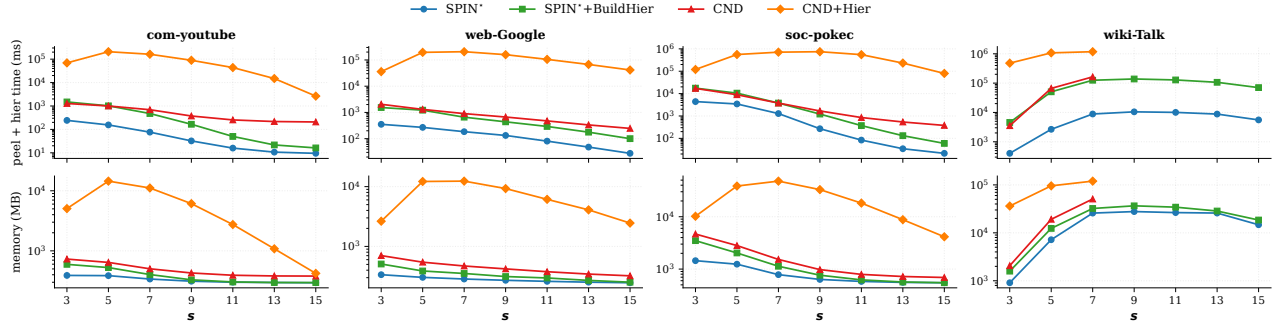


Figure 7: Hierarchy bench on four graphs: wall time (top) and peak memory (bottom).

higher- r nuclei on the evaluated metrics at much lower cost; we scope this claim to the datasets and metrics below.

Setup. For cross- r comparisons we use the standard (r, s) -nucleus parameterisation [39]; the paper’s algorithmic results apply only to $r=1$, while the $r=2$ and $r=3$ scores serve as baselines from the same family. Edge- and triangle-level scores must be compared with vertex rankings, so we project them by assigning each vertex the maximum score of an incident r -clique, then rank descending with ties broken uniformly.

The ground-truth datasets are com-dblp and com-youtube from SNAP [48]. com-dblp has 317,080 vertices, 1,049,866 edges, and 5,000 venue-based ground-truth communities. com-youtube has 1,134,890 vertices, 2,987,624 edges, and 5,000 ground-truth communities.

We use three ranking metrics. Precision@ K is the fraction of top- K vertices that belong to at least one ground-truth community. rec50 is the number of communities with at least 50% of their vertices inside the top- K . Triangle density is the number of triangles per vertex in the induced top- K subgraph.

Case study 1: s is a granularity knob. k -core ($s=2$) is known to be too coarse on dense graphs: 80–100% of vertices share a handful of small core values, so the ranking cannot separate dense pockets from the bulk [39, 40]. The (r, s) -nucleus generalisation [39] fixes this by replacing edge-support with s -clique-support; for $r=1$, the support of v is the number of s -cliques containing v . Table 6 measures the resulting granularity on com-dblp by sweeping s from 2 to 30. “Nonzero $|V|$ ” is the alive set (vertices with $k_{1,s}(v) \geq 1$), “max core” is the largest attained core, and “top- K min” is the smallest core value among the top- K ranked vertices, i.e. the threshold that separates the top- K from the bulk.

Table 6: Active set on com-dblp as s grows.

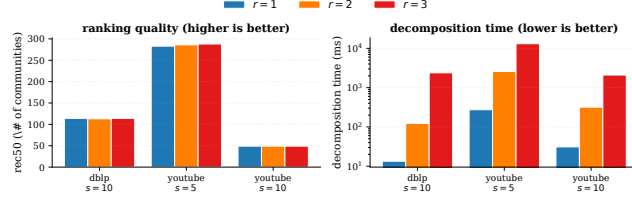
s	nonzero $ V $	% of n	max core	top-100 min	top-1k min
2	317,080	100.00	1.13×10^2	1.13×10^2	3.10×10^1
3	269,181	84.89	6.33×10^3	6.33×10^3	4.65×10^2
4	194,957	61.49	2.34×10^5	2.34×10^5	4.50×10^3
5	127,805	40.31	6.44×10^6	6.44×10^6	3.15×10^4
8	36,412	11.48	3.86×10^{10}	3.86×10^{10}	2.63×10^6
10	19,354	6.10	5.97×10^{12}	5.97×10^{12}	2.02×10^7
15	6,767	2.13	2.74×10^{17}	2.74×10^{17}	2.65×10^8
20	3,396	1.07	1.69×10^{21}	1.69×10^{21}	1.41×10^8
25	2,069	0.65	2.18×10^{24}	2.18×10^{24}	2.63×10^6
30	1,358	0.43	7.61×10^{26}	7.61×10^{26}	4.65×10^2

At $s=2$ (k -core), the alive set is the whole graph and max core is only 113, so the entire ranking compresses into a two-digit dynamic range, exactly the collapse [39] observed. Each step up in s enlarges the dynamic range and shrinks the alive set monotonically: max core grows from 113 at $s=2$ to 6.3×10^3 at $s=3$ (56 $\times$), to 6.0×10^{12} at $s=10$, and to 7.6×10^{26} at $s=30$ (ten-plus orders of magnitude). The top- K thresholds widen in step: at $s=10$ the top-100 vertices all have core $\geq 6.0 \times 10^{12}$ while the rest of the alive set drops below 2.0×10^7 , a five-decade gap that makes the top-100 unambiguously identifiable. The alive set shrinks 234 $\times$ ($317k \rightarrow 1,358$) across the same sweep. The headline is that the same $r=1$ primitive serves analytics tasks at different scales: $s \leq 3$ gives a community-scale ranking compatible with the ground-truth recall metric used in Case study 2; $s \in [5, 10]$ isolates the few thousand densest vertices, the natural prefix size for downstream dense-subgraph and team-formation tasks [13, 18, 19]; $s \geq 15$ shrinks to a few thousand candidates suitable for influence-maximisation seeding [11, 24]. s is the knob that picks the regime.

Case study 2: $(1, s)$ improves the k -core ranking. On com-dblp, the first non-trivial clique support already improves the ground-truth ranking over k -core. Table 7 reports top-10,000 quality.

Table 7: Top- K ranking quality on com-dblp.

Method	top- K	Precision	rec50	tri/v
k -core	10,000	0.504	94	112.3
(1, 3)-core	10,000	0.523	114	113.6
(1, 5)-core	10,000	0.523	114	113.5
(1, 10)-core	10,000	0.523	114	113.4
(1, 15)-core	6,767	0.548	65	154
(1, 30)-core	1,358	0.507	4	523

**Figure 8: Cross- r comparison on com-dblp and com-youtube. Quality (left) is nearly identical across r ; time (right, log scale) is 1–3 orders of magnitude smaller at $r=1$.**

(1, 3)-core raises precision from 0.504 to 0.523 and rec50 from 94 to 114. For $s \in \{3, 5, 10\}$, the top-10,000 quality is identical in this table. For larger s , the active set drops below 10,000, so the comparison changes from ranking quality at fixed K to density of the surviving subgraph. (1, 30)-core reaches 523 triangles per vertex on 1,358 vertices.

Case study 3: higher r does not buy quality on the studied graphs. Edge- and triangle-level nuclei sit between (1, s) and explicit clique enumeration on the cost axis; whether they buy quality at that cost is an open question on the same datasets. Fixing s and comparing $r \in \{1, 2, 3\}$ across com-dblp ($s=10$) and com-youtube ($s \in \{5, 10\}$), Figure 8 shows the answer at a glance: within each cell the three quality bars are nearly equal (left panel), while the time bars span 1–3 orders of magnitude (right panel, log scale). On com-dblp at $s=10$, (1, 10) and (3, 10) agree on rec50 (114 vs. 114) while $r=1$ is 178× faster (13.4 ms vs. 2380 ms); on com-youtube the same pattern repeats at every s we compared, with $r=1$ within 5 rec50 points of $r=3$ (at $s=5$: 283 vs. 288; at $s=10$: 49 vs. 49) and between 9.8× and 68× faster than $r=3$ (and 6.2–10.3× faster than $r=2$). $r=1$ therefore gives the best observed quality–runtime tradeoff for vertex ranking on these graphs; this does not prove higher r is never useful, only that increasing r here mostly adds cost.²

Case study 4: identifying ego network on DBLP, with the (r, s) grid on Han’s 1-hop ego. We build a fresh coauthor graph from the official DBLP XML dump (April 2024, 3.4M authors and 23.0M coauthorship edges), anchor on Jiawei Han (DBLP key Jiawei Han 0001), and take his 1-hop coauthor ego (1,358 vertices). Figure 9 decomposes the ego under four settings $(r, s) \in \{(1, 4), (3, 4), (1, 10), (3, 10)\}$, with (3, s) obtained via the NuclearCD [49] reference implementation. For each (r, s) we partition the alive subgraph (vertices with $k_{r,s}(v) \geq 1$; for $r=3$ we project to vertices via the max containing-triangle’s (3, s) core) into modules using greedy near-clique extraction (size ≥ 25 , internal density ≥ 0.25 , which is looser than NuclearCD CS-II so that both tight and field-scale loose groups surface). For each module C we report a semantic label, member count, the number of ≥ 5 -co-author collaborative papers, and the active ratio (fraction of members appearing in at least one such

paper), following NuclearCD [49] but with a stricter ≥ 5 within-module co-author threshold so loose modules at small s remain discriminable from tight modules at large s .

Increasing s tightens the modules and lifts the active ratio toward 100%.

At (1, 4) the alive subgraph contains 1,323 vertices and the loose partition surfaces 6 modules, ranging from a 150-member loose IR/Web cluster (density 0.43, active 93%) down to two tighter 32- and 37-vertex groups (NLP and a Data Mining sub-team) whose active ratios drop to 69% and 76%; this is the long-tail behaviour NuclearCD [49] observed. At (1, 10) the alive subgraph shrinks to 590 vertices, and the same partition surfaces only 3 modules, all densities 0.74–0.94 and active ratios 85%–97%: ML Safety (32, 97%), Data Mining (49, 96%), IR/Web (26, 85%). Raising s therefore filters out loose, weakly-collaborating groups and retains compact, strongly-collaborating teams: the size-weighted average active ratio rises from 91% at $s=4$ to 93% at $s=10$ in our setting, and would saturate further at $s=15$ or higher (a few of the $s=4$ modules are already at 100%).

Raising r from 1 to 3 does not change the result. At each s the (1, s) and (3, s) panels are pairwise identical (same module count, sizes, labels, representative members, and active ratios), even though the per-vertex absolute core thresholds differ by orders of magnitude. The two decompositions *rank* vertices the same way on this ego, so the more expensive triangle-centric $r=3$ delivers no extra resolution beyond what the cheap vertex-centric $r=1$ already provides. This is the broader picture in Table ??: (1, s) achieves community-detection quality matching higher- r baselines while being substantially cheaper to compute (Tables ??, 5).

Takeaway. On the DBLP and YouTube community-ranking tasks, $r=1$ matches higher- r quality metrics at much lower cost; the ego-network study (Figure 9) shows the same decomposition also yields interpretable local structure. The evidence is scoped to these datasets and metrics rather than a universal claim, but it is enough to justify optimising the exact $r=1$ case.

10 Related Work

This paper uses prior ideas on core peeling, compact clique representations, local fixed-point characterisations, parallel construction, and hierarchy output; the new ingredient is the static-CPI invariant for $r=1$.

Enumeration-based core and nucleus decomposition. The k -core was introduced by Seidman [41] and is supported by a long line of efficient algorithms [5, 6, 8, 25, 35, 36, 45–47, 50]. Higher-order extensions strengthen the edge witness: k -truss [1, 10, 12, 22] uses triangle support, and *nucleus decomposition* [39, 40] generalises the witness to arbitrary s -cliques by peeling r -cliques according to their support in s -cliques. Hypergraph generalisations adopt the same peeling paradigm under different cohesion definitions [4, 7, 26, 29]. We specialise to the vertex-centric $r=1$ case and exploit a state invariant that does not hold for general r .

Compact clique representations. Bron–Kerbosch search [9], Tomita pivoting [44], and degeneracy ordering [15] are standard tools for clique enumeration; Pivoter’s succinct clique tree compactly encodes clique counts [23], and the Clique Path Index extends this line to support general nucleus decomposition with mutating hold/pivot updates during peeling [49]. The static-CPI counter form

²Above $s=10$ on com-youtube the active set falls below the smallest SNAP community size (e.g., 95 vertices at $s=15$), so rec50 becomes uninformative for any r ; we cap the cross- r comparison at $s=10$.

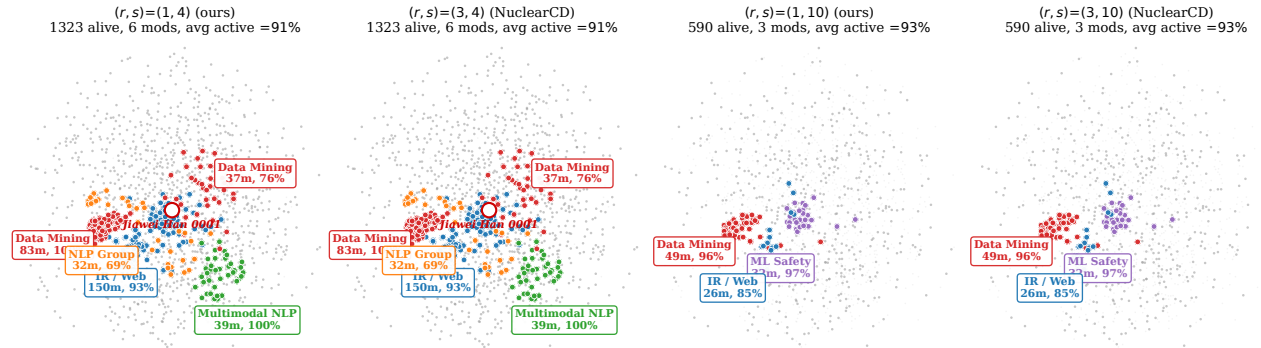


Figure 9: Han's 1-hop coauthor ego under $(r, s) \in \{(1, 4), (3, 4), (1, 10), (3, 10)\}$. Raising r does not change the result; raising s filters out loose modules and lifts the average active ratio.

of this paper shows that for vertex peeling these path edits never need to be materialised.

Local and parallel peeling. h -index characterisations of core values [21] were studied for local core computation [31, 38] and extended to distributed and parallel settings [35, 50]. In this paper, the fixed-point view is adapted to the implicit s -cliques encoded by the CPI, and SPIN* provides its peel-order realisation. The difference from prior local h -index work is not the operator but the representation: the supporting cliques are held in static CPI paths and updated by binomial support-loss formulas.

Parallel construction and clique counting. Parallel clique search typically distributes the recursive enumeration work; ParaBuild follows the same seed-level decomposition. Because the downstream peel never mutates path sets, construction partitions into thread-independent subproblems whose results combine by a deterministic merge; this is an engineering consequence of the $r=1$ state model rather than a new clique-enumeration paradigm.

Hierarchical cohesive subgraphs. Nucleus decompositions naturally form hierarchies across core levels [37, 39, 40, 42, 43], and elder-rule join-tree extraction is standard in computational topology [14]. Prior nucleus pipelines extract such hierarchies by sorting the decomposition output or revisiting the cliques; BuildHier instead reuses the preserved static CPI, treating each leaf as a hyper-edge born at its minimum core value, so the hierarchy is a byproduct of preserving the CPI through peeling.

Applications of k -core and nucleus decomposition. Core and nucleus decompositions are workhorses for community detection [13, 17, 18, 20], densest-subgraph discovery [19], influential-vertex and role analysis [11, 24, 30], network visualization [51], and downstream uses in social analysis [3, 16], biological networks [28, 33], complex-network analysis [27], anomaly and security applications [2, 34], and recommendation [32]. This paper inherits the same downstream uses; the contribution is making the vertex-centric $(1, s)$ output cheaper to obtain at higher s .

Empirical graph mining methodology. The evaluation follows common practice in cohesive-subgraph experiments: SNAP community ground truth, large public graphs, and runtime and memory comparisons [40, 48, 49].

11 Conclusion

For $(1, s)$ -nucleus decomposition, vertex peeling preserves every edge among surviving vertices. The path structure of the CPI therefore stays valid throughout the peel, and exact decomposition reduces to counter maintenance over a static symbolic index. SPIN* realises this fixed point in deletion order, ParaBuild parallelises the construction that is now the bottleneck, and BuildHier recovers the nucleus hierarchy from the static index. The matched configurations agree with the mutable baseline while reducing runtime and memory on the evaluated benchmarks, and the case studies show that vertex-centric $(1, s)$ matches higher- r quality metrics on community tasks at a fraction of the cost. The static-CPI invariant is specific to $r=1$; extending it to restricted $r \geq 2$ settings, to streaming and dynamic graphs, and to GPU/distributed execution are the natural next steps.

References

- [1] Esra Akbas and Peixiang Zhao. 2017. Truss-based community search: a truss-equivalence based indexing approach. *Proc. VLDB Endow.* 10, 11 (Aug. 2017), 1298–1309. <https://doi.org/10.14778/3137628.3137640>
- [2] Leman Akoglu, Hanghang Tong, and Danai Koutra. 2015. Graph based anomaly detection and description: A survey. *Data Mining and Knowledge Discovery* 29, 3 (2015), 626–688. <https://doi.org/10.1007/s10618-014-0365-y>
- [3] J. Ignacio Alvarez-Hamelin, Luca Dall'Asta, Alain Barrat, and Alessandro Vespignani. 2005. Large scale networks fingerprinting and visualization using the k -core decomposition. In *Proceedings of the 18th International Conference on Neural Information Processing Systems* (Vancouver, British Columbia, Canada) (NIPS'05). MIT Press, Cambridge, MA, USA, 41–50.
- [4] Naheed Anjum Arafat, Arijit Khan, Arpit Kumar Rai, and Bishwamitra Ghosh. 2023. Neighborhood-Based Hypergraph Core Decomposition. *Proc. VLDB Endow.* 16, 9 (2023), 2061–2074. <https://doi.org/10.14778/3598581.3598582>
- [5] Vladimir Batagelj and Matjaž Zaveršnik. 2003. An $O(m)$ Algorithm for Cores Decomposition of Networks. *arXiv:cs/0310049* [cs.DS]
- [6] Vladimir Batagelj and Matjaž Zaveršnik. 2011. Fast algorithms for determining (generalized) core groups in social networks. *Adv. Data Anal. Classif.* 5, 2 (July 2011), 129–145. <https://doi.org/10.1007/s11634-010-0079-y>
- [7] Ginestra Bianconi and Sergey N Dorogovtsev. 2024. Nature of hypergraph k -core percolation problems. *Physical Review E* 109, 1 (2024), 014307. <https://doi.org/10.1103/PhysRevE.109.014307>
- [8] Francesco Bonchi, Francesco Gullo, Andreas Kaltenbrunner, and Yana Volkovich. 2014. Core decomposition of uncertain graphs. In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (New York, New York, USA) (KDD '14). Association for Computing Machinery, New York, NY, USA, 1316–1325. <https://doi.org/10.1145/2623330.2623655>
- [9] Coen Bron and Joep Kerbosch. 1973. Algorithm 457: finding all cliques of an undirected graph. *Commun. ACM* 16, 9 (Sept. 1973), 575–577. <https://doi.org/10.1145/362342.362367>
- [10] Yulin Che, Zhuohang Lai, Shixuan Sun, Yue Wang, and Qiong Luo. 2020. Accelerating truss decomposition on heterogeneous processors. *Proc. VLDB Endow.* 13,

- 10 (June 2020), 1751–1764. <https://doi.org/10.14778/3401960.3401971>
- [11] Duanbing Chen, Linyuan Lü, Ming-Sheng Shang, Yi-Cheng Zhang, and Tao Zhou. 2012. Identifying influential nodes in complex networks. *Physica A: Statistical Mechanics and its Applications* 391, 4 (2012), 1777–1787. <https://doi.org/10.1016/j.physa.2011.09.017>
- [12] Jonathan Cohen. 2008. *Trusses: Cohesive Subgraphs for Social Network Analysis*. Technical Report. National Security Agency.
- [13] Wanyun Cui, Yanghua Xiao, Haixun Wang, and Wei Wang. 2014. Local search of communities in large graphs. In *International Conference on Management of Data, SIGMOD 2014, Snowbird, UT, USA, June 22–27, 2014*, Curtis E. Dyreson, Feifei Li, and M. Tamer Özsu (Eds.). ACM, New York, NY, USA, 991–1002. <https://doi.org/10.1145/2588555.2612179>
- [14] Herbert Edelsbrunner and John Harer. 2010. *Computational Topology: An Introduction*. American Mathematical Society, Providence, RI, USA. <https://doi.org/10.1090/mbk/069>
- [15] David Eppstein, Maarten Löffler, and Darren Strash. 2010. Listing All Maximal Cliques in Sparse Graphs in Near-Optimal Time. In *Proc. 21st International Symposium on Algorithms and Computation (ISAAC) (Lecture Notes in Computer Science)*, Vol. 6506. Springer, Berlin, Heidelberg, 403–414. https://doi.org/10.1007/978-3-642-17517-6_36
- [16] Ernesto Estrada and Juan A. Rodriguez-Velazquez. 2005. Complex networks as hypergraphs. *arXiv:physics/0505137 [physics.soc-ph]*
- [17] Santo Fortunato. 2010. Community detection in graphs. *Physics Reports* 486, 3–5 (2010), 75–174. <https://doi.org/10.1016/j.physrep.2009.11.002>
- [18] Christos Giatsidis, Dimitrios M. Thilikos, and Michalis Vazirgiannis. 2011. Evaluating Cooperation in Communities with the k-Core Structure. In *International Conference on Advances in Social Networks Analysis and Mining, ASONAM 2011, Kaohsiung, Taiwan, 25–27 July 2011*. IEEE Computer Society, Washington, DC, USA, 87–93. <https://doi.org/10.1109/ASONAM.2011.65>
- [19] Aristides Gionis and Charalampos E. Tsourakakis. 2015. Dense Subgraph Discovery: KDD 2015 tutorial. In *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Sydney, NSW, Australia, August 10–13, 2015*, Longbing Cao, Chengqi Zhang, Thorsten Joachims, Geoffrey I. Webb, Dragos D. Margineantu, and Graham Williams (Eds.). ACM, New York, NY, USA, 2313–2314. <https://doi.org/10.1145/2783258.2789987>
- [20] M. Girvan and M. E. J. Newman. 2002. Community structure in social and biological networks. *Proceedings of the National Academy of Sciences* 99, 12 (2002), 7821–7826. <https://doi.org/10.1073/pnas.122653799> <https://www.pnas.org/doi/pdf/10.1073/pnas.122653799>
- [21] J E Hirsch. 2005. An index to quantify an individual's scientific research output. *Proc Natl Acad Sci U S A* 102, 46 (2005), 16569–16572. <https://doi.org/10.1073/pnas.0507655102>
- [22] Xin Huang, Hong Cheng, Lu Qin, Wentao Tian, and Jeffrey Xu Yu. 2014. Querying k-truss community in large and dynamic graphs. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data (Snowbird, Utah, USA) (SIGMOD '14)*. Association for Computing Machinery, New York, NY, USA, 1311–1322. <https://doi.org/10.1145/2588555.2610495>
- [23] Shweta Jain and C. Seshadhri. 2020. The Power of Pivoting for Exact Clique Counting. In *Proceedings of the 13th International Conference on Web Search and Data Mining (Houston, TX, USA) (WSDM '20)*. Association for Computing Machinery, New York, NY, USA, 268–276. <https://doi.org/10.1145/3336191.3371839>
- [24] David Kempe, Jon Kleinberg, and Éva Tardos. 2003. Maximizing the spread of influence through a social network. In *Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (Washington, D.C.) (KDD '03)*. Association for Computing Machinery, New York, NY, USA, 137–146. <https://doi.org/10.1145/956750.956769>
- [25] Wissam Khauuid, Marina Barsky, Venkatesh Srinivasan, and Alex Thomo. 2015. K-core decomposition of large networks on a single PC. *Proc. VLDB Endow.* 9, 1 (2015), 13–23. <https://doi.org/10.14778/2850469.2850471>
- [26] Dahee Kim, Junghoon Kim, Sungsu Lim, and Hyun Ji Jeong. 2023. Exploring Cohesive Subgraphs in Hypergraphs: The (k,g)-Core Approach. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (Birmingham, United Kingdom) (CIKM '23)*. Association for Computing Machinery, New York, NY, USA, 4013–4017. <https://doi.org/10.1145/3583780.3615275>
- [27] Maksim Kitsak, Lazaros K Gallos, Shlomo Havlin, Fredrik Liljeros, Lev Muchnik, H Eugene Stanley, and Hernán A Makse. 2010. Identification of influential spreaders in complex networks. *Nature physics* 6, 11 (2010), 888–893. <https://doi.org/10.1038/nphys1746>
- [28] Steffen Klamt, Utz-Uwe Haus, and Fabian Theis. 2009. Hypergraphs and Cellular Networks. *PLOS Computational Biology* 5, 5 (05 2009), e1000385. <https://doi.org/10.1371/journal.pcbi.1000385>
- [29] Jongshin Lee, Kwang-Il Goh, Deok-Sun Lee, and B. Kahng. 2023. (k,q)-core decomposition of hypergraphs. *Chaos, Solitons & Fractals* 173 (2023), 113645. <https://doi.org/10.1016/j.chaos.2023.113645>
- [30] Linyuan Lü, Duanbing Chen, Xiao-Long Ren, Qian-Ming Zhang, Yi-Cheng Zhang, and Tao Zhou. 2016. Vital nodes identification in complex networks. *Physics Reports* 650 (2016), 1–63. <https://doi.org/10.1016/j.physrep.2016.06.007>
- [31] Linyuan Lu, Tao Zhou, Qian-Ming Zhang, and H. Eugene Stanley. 2016. The h-Index of a Network Node and Its Relation to Degree and Coreness. *Nature Communications* 7 (2016), 10168. <https://doi.org/10.1038/ncomms10168>
- [32] Mingsong Mao, Jie Lu, Jialin Han, and Guangquan Zhang. 2019. Multiobjective e-commerce recommendations based on hypergraph ranking. *Information Sciences* 471 (2019), 269–287. <https://doi.org/10.1016/j.ins.2018.07.029>
- [33] Sergei Maslov and Kim Sneppen. 2002. Specificity and stability in topology of protein networks. *Science* 296, 5569 (2002), 910–913. <https://doi.org/10.1126/science.1065103>
- [34] Juan Manuel Matalobos Veiga, Regino Criado, Miguel Romance Del Rio, Sergio Iglesias Perez, Alberto Partida Rodriguez, and Karan Kabbur Hanumanthappa Manjunatha. 2024. A Hypergraph-based Model for Cyberincident-related Data Analysis. In *Proceedings of the 2024 European Interdisciplinary Cybersecurity Conference (Xanthi, Greece) (EICC '24)*. Association for Computing Machinery, New York, NY, USA, 161–162. <https://doi.org/10.1145/3655693.3661300>
- [35] Alberto Montresor, Francesco De Pellegrini, and Daniele Miorandi. 2013. Distributed k-Core Decomposition. *IEEE Transactions on Parallel and Distributed Systems* 24, 2 (2013), 288–300. <https://doi.org/10.1109/TPDS.2012.124>
- [36] Ahmet Erdem Sariyüce, Buğra Gedik, Gabriela Jacques-Silva, Kun-Lung Wu, and Ümit V. Çatalyürek. 2013. Streaming algorithms for k-core decomposition. *Proc. VLDB Endow.* 6, 6 (2013), 433–444. <https://doi.org/10.14778/2536336.2536344>
- [37] Ahmet Erdem Sariyüce and Ali Pinar. 2016. Fast hierarchy construction for dense subgraphs. *Proc. VLDB Endow.* 10, 3 (Nov. 2016), 97–108. <https://doi.org/10.14778/3021924.3021927>
- [38] Ahmet Erdem Sariyüce, C. Seshadhri, and Ali Pinar. 2018. Local algorithms for hierarchical dense subgraph discovery. *Proc. VLDB Endow.* 12, 1 (Sept. 2018), 43–56. <https://doi.org/10.14778/3275536.3275540>
- [39] Ahmet Erdem Sariyüce, C. Seshadhri, Ali Pinar, and Ümit V. Çatalyürek. 2015. Finding the Hierarchy of Dense Subgraphs Using Nucleus Decompositions. In *Proc. International Conference on World Wide Web (WWW)*. Association for Computing Machinery, New York, NY, USA, 927–937. <https://doi.org/10.1145/2736277.2741640>
- [40] Ahmet Erdem Sariyüce, C. Seshadhri, Ali Pinar, and Ümit V. Çatalyürek. 2017. Nucleus Decompositions for Identifying Hierarchy of Dense Subgraphs. *ACM Trans. Web* 11, 3, Article 16 (July 2017), 27 pages. <https://doi.org/10.1145/3057742>
- [41] Stephen B. Seidman. 1983. Network Structure and Minimum Degree. *Social Networks* 5, 3 (1983), 269–287. [https://doi.org/10.1016/0378-8733\(83\)90028-X](https://doi.org/10.1016/0378-8733(83)90028-X)
- [42] Jessica Shi, Laxman Dhulipala, and Julian Shun. 2021. Theoretically and practically efficient parallel nucleus decomposition. *Proc. VLDB Endow.* 15, 3 (Nov. 2021), 583–596. <https://doi.org/10.14778/3494124.3494140>
- [43] Jessica Shi, Laxman Dhulipala, and Julian Shun. 2024. Parallel Algorithms for Hierarchical Nucleus Decomposition. *Proc. ACM Manag. Data* 2, 1, Article 32 (Feb. 2024), 27 pages. <https://doi.org/10.1145/3639287>
- [44] Etsuji Tomita, Akira Tanaka, and Haruhisa Takahashi. 2006. The Worst-Case Time Complexity for Generating All Maximal Cliques and Computational Experiments. *Theoretical Computer Science* 363, 1 (2006), 28–42. <https://doi.org/10.1016/j.tcs.2006.06.015>
- [45] Dong Wen, Lu Qin, Ying Zhang, Xuemin Lin, and Jeffrey Xu Yu. 2016. I/O efficient core graph decomposition at web scale. In *2016 IEEE 32nd International Conference on Data Engineering (ICDE)*. IEEE, Piscataway, NJ, USA, 133–144. <https://doi.org/10.1109/ICDE.2016.7498235>
- [46] Dong Wen, Lu Qin, Ying Zhang, Xuemin Lin, and Jeffrey Xu Yu. 2019. I/O Efficient Core Graph Decomposition: Application to Degeneracy Ordering. *IEEE Trans. Knowl. Data Eng.* 31, 1 (2019), 75–90. <https://doi.org/10.1109/TKDE.2018.2833070>
- [47] Dong Wen, Bohua Yang, Lu Qin, Ying Zhang, Lijun Chang, and Rong-Hua Li. 2022. Computing K-Cores in Large Uncertain Graphs: An Index-Based Optimal Approach. *IEEE Transactions on Knowledge and Data Engineering* 34, 7 (2022), 3126–3138. <https://doi.org/10.1109/TKDE.2020.3023925>
- [48] Jaewon Yang and Jure Leskovec. 2015. Defining and Evaluating Network Communities Based on Ground-Truth. *Knowledge and Information Systems* 42, 1 (2015), 181–213. <https://doi.org/10.1007/s10115-013-0693-z>
- [49] Wenqian Zhang, Zhengyi Yang, Dong Wen, Yi Ding, Wenjie Zhang, and Xuemin Lin. 2026. Nucleus Decomposition Revisited: An Efficient Counting-Based Approach. *Proc. ACM Manag. Data* 4, 3, Article 215 (May 2026), 26 pages. <https://doi.org/10.1145/3802092>
- [50] Wenqian Zhang, Zhengyi Yang, Dong Wen, and Xiaoyang Wang. 2023. Efficient Distributed Core Graph Decomposition. In *2023 IEEE International Conference on Data Mining Workshops (ICDMW)*. IEEE, Piscataway, NJ, USA, 1023–1031. <https://doi.org/10.1109/ICDMW60847.2023.00135>
- [51] Feng Zhao and Anthony K. H. Tung. 2012. Large scale cohesive subgraphs discovery for social network visual analysis. *Proc. VLDB Endow.* 6, 2 (2012), 85–96. <https://doi.org/10.14778/2535568.2448942>